

Image-Processing Audit

An algorithmic & colorimetric correctness review of *latent*, a RAW photo developer, verified against primary literature and standards

Code & Image-Processing Audit

2026-06-27

Contents

About this document	2
1 Color Science Audit — latent	3
1.1 1. Summary — what the code does and its colorimetric model	3
1.2 2. Point-by-point verification	4
1.3 3. Findings	9
1.4 4. References	12
2 Audit 02 — RAW Decode & Demosaic	13
2.1 1. Summary of the Decode / Demosaic Model	13
2.2 2. Point-by-Point Verification	14
2.3 3. Findings by Severity	22
2.4 4. Verdict Summary	24
2.5 5. References	24
3 Audit 03 — Tone & Color-Grading Math	26
3.1 1. Summary of the Tone / Color Model	26
3.2 2. Point-by-Point Verification	27
3.3 3. Findings by Severity	35
3.4 4. References	39
4 Audit 04 — Spatial & Frequency-Domain Filters	41
4.1 1. Summary of Each Filter’s Model	41
4.2 2. Point-by-Point Verification	43
4.3 3. Findings by Severity	51
4.4 4. Verdict Summary	58
4.5 5. References	58
5 Audit 05 — Geometry & Optics	60
5.1 1. Summary of the Geometry Model and the Single-Resample Design . .	60
5.2 2. Point-by-Point Verification	61
5.3 3. Findings by Severity	70
5.4 4. References	72

About this document

This is the consolidated, formula-typeset edition of the five image-processing audit notes for **latent**, a RAW photo-development application written in Rust. Each chapter audits one stage of the rendering pipeline against the canonical paper, standard, or reference implementation for the algorithm in question — the sources are downloaded under docs/ in the repository and cited inline. A companion software **code review** and an **executive summary** (with a de-duplicated, severity-ranked findings register) live alongside this file in audits/.

Verdicts use the scale **Correct** / **Correct-with-caveats** / **Questionable** / **Incorrect**; findings use **Critical** / **High** / **Medium** / **Low** / **Note**. The baseline build was clean (cargo fmt, clippy, and 50 tests all pass) before any review began, so everything here is *beyond* what the compiler and linter catch.

Chapter 1

Color Science Audit — latent

Scope: the color-science math of the latent RAW developer. **Audited files:**

- `latent-image/src/color.rs` (matrix ops, primaries, working-space construction, luma, HSV)
- `latent-export/src/lib.rs` (sRGB OETF, highlight rolloff, output transform, ICC)
- `latent-raw/src/lib.rs` (`color_matrix()`, `cam_xyz/cam_mul` feed; decode order)
- `latent-app/src/main.rs` (pipeline ordering, for context)

Date: 2026-06-27 **Method:** every numeric claim was re-derived from first principles (pure-Python reference implementation of the same matrix algebra) and checked against primary standards (IEC 61966-2-1, ISO 22028-2 / ROMM RGB, Adobe DNG 1.4.0.0, SMPTE RP 177). Drift figures are reported in both 8-bit and 16-bit code units.

1.1 1. Summary — what the code does and its colorimetric model

latent develops RAW files through a **fixed linear-light pipeline** whose decode order is (`latent-app/src/main.rs:63-70`):

normalize → white balance (*mosaic*, *cam_mul*) → demosaic → highlight reconstruct → $\mathbf{M}_{\text{cam} \rightarrow \text{work}}$

The **working space** is *linear ProPhoto/ROMM primaries pinned to a D65 white point* (`color.rs:125-136`). Camera RGB is lifted into XYZ by inverting the file's `cam_xyz` (DNG ColorMatrix, an XYZ→camera matrix), then mapped XYZ→working. At export (`latent-export/src/lib.rs`), working RGB is taken to **linear sRGB** by a row-normalized product of the working→XYZ matrix and the published IEC XYZ→sRGB matrix, passed through a **highlight rolloff** (Reinhard-style knee at 0.98), then the **sRGB OETF**, then quantized to 8/16-bit with an embedded sRGB ICC profile.

The colorimetric model is internally **D65-consistent end to end**: camera matrix inverse, working primaries, and sRGB output all sit at D65, so no chromatic adaptation

is performed anywhere — by design. Two deliberate engineering choices distinguish it from a textbook ICC pipeline: (a) the working space is *non-standard* (ProPhoto primaries at D65, not the ISO-22028-2 D50 white), and (b) **row-normalization** is used twice to pin the neutral axis in place of an explicit chromatic-adaptation transform.

1.2 2. Point-by-point verification

#	Claim under test	Verdict	Authority
1	XYZ_T0_LINEAR_SRGB = the IEC sRGB matrix (4-decimal)	Correct-with-caveats	IEC 61966-2-1 / Amd.1
2	ProPhoto primaries pinned to D65 is a sound, adaptation-free working space	Correct-with-caveats	ISO 22028-2 (ROMM)
3	rgb_to_xyz construction (primaries-as-columns, scaled to white)	Correct	SMPTE RP 177 / Lindbloom
4	Row-normalizing camera_to_working is the right way to avoid double-WB	Questionable	Adobe DNG 1.4.0.0 §6
4b	Row-normalizing working→sRGB; “~1e-4” chromatic drift claim	Correct-with-caveats	(numerical)
5	LUMA_WEIGHTS = Y row of ProPhoto-D65 matrix; blue ≈ 0 by design	Correct-with-caveats	(derived); Rec. 709
6	sRGB OETF constants (12.92, 0.0031308, 1.055, 1/2.4, 0.055)	Correct	IEC 61966-2-1
7	highlight_rolloff knee 0.98, white→254/255	Correct (by design)	Reinhard / display-referred practice
8	cam_xyz is XYZ→camera; inverse is camera→XYZ	Correct	Adobe DNG 1.4.0.0 §6
9	No Bradford chromatic adaptation needed	Correct-with-caveats	DNG §6 / ICC practice

1.2.1 2.1 (#1) The XYZ→linear-sRGB matrix — color.rs:89-93

The constant is

$$\mathbf{M}_{\text{XYZ} \rightarrow \text{sRGB}} = \begin{bmatrix} 3.2406 & -1.5372 & -0.4986 \\ -0.9689 & 1.8758 & 0.0415 \\ 0.0557 & -0.2040 & 1.0570 \end{bmatrix}.$$

This is **bit-exact** with the matrix printed in IEC 61966-2-1 (reproduced verbatim in the freely-available bg-sRGB amendment text, “*Conversion from XYZ (D65) to bg-sRGB*”).

So the code transcribes the standard correctly.

Caveat (precision). The standard explicitly labels this 4-decimal matrix as accurate *only to 8-bit samples*; the **2003 Amendment 1** supplies a 7-decimal matrix (3.2406255, -1.5372080, -0.4986286; ...) recommended for 16-bit. Re-deriving the matrix at full precision from the Rec. 709 primaries gives

$$\mathbf{M}_{\text{XYZ} \rightarrow \text{sRGB}}^{\text{exact}} = \begin{bmatrix} 3.24096994 & -1.53738318 & -0.49861076 \\ -0.96924364 & 1.87596750 & 0.04155506 \\ 0.05563008 & -0.20397696 & 1.05697151 \end{bmatrix},$$

which differs from the code’s constant by up to 3.7×10^{-4} (element [0][2]/[2][2]). That is ≈ 0.094 of an 8-bit code (invisible) but ≈ 24 of a 16-bit code (real, though still well under any perceptual threshold). Because latent advertises a true-16-bit export path (save_16), using the 4-decimal constant is a minor precision leak on the 16-bit output. Verified directly: $\mathbf{M}_{\text{code}} \cdot [0.95047, 1, 1.08883]^T = [1.000002, 1.000076, 0.999834]^T$ (should be exactly $[1, 1, 1]$), confirming the rounded matrix does not map D65 white perfectly to neutral.

1.2.2 2.2 (#2) ProPhoto primaries at D65 — color.rs:125-136

The primaries

$$R(0.7347, 0.2653), \quad G(0.1596, 0.8404), \quad B(0.0366, 0.0001)$$

match ISO 22028-2 (ROMM RGB) exactly (the registry spec lists the same numbers). However, the same spec states unambiguously: “*The reference white for ROMM RGB is specified as D50 ($x=0.3457$, $y=0.3585$).*” The code pins these primaries to **D65** (D65_WHITE = [0.3127, 0.3290]).

This is a non-standard space. It is *not* ProPhoto RGB / ROMM RGB; it is “ROMM primaries at D65,” which has no ICC/ISO identity. The code comment calls it “linear ProPhoto primaries at D65,” which is an accurate description, but a reader should not assume interoperability with anything tagged ProPhoto.

Is the “no chromatic adaptation needed” reasoning sound? **Yes, internally.** Because the camera→XYZ inverse, the working white, and the sRGB output white are *all* D65, the white point never changes across the pipeline, so a Bradford CA between stages would be the identity. Pinning the working white to D65 is a legitimate way to make the whole chain adaptation-free. The cost is purely that the working space is unconventional, not that it is colorimetrically broken: any XYZ color is still represented exactly (the primaries are real, the matrix is invertible and round-trips to $< 10^{-5}$, verified). The Y row at D65 is [0.27882, 0.72107, 0.00011] vs. the standard D50 ProPhoto Y row [0.28804, 0.71188, 0.0000857] — same character (huge green, near-zero blue), shifted only by the white move.

Derived ROMM-at-D50 RGB→XYZ matches Bruce Lindbloom’s published ProPhoto-D50 matrix to 2.2×10^{-5} , confirming the construction is faithful; the only deviation from the standard is the deliberate D50→D65 white substitution.

1.2.3 2.3 (#3) The rgb_to_xyz construction — color.rs:107-123

The routine builds each primary's XYZ at unit luminance, $[x/y, 1, (1-x-y)/y]$, places the three as **columns** of a basis $\mathbf{B}$, then solves $\mathbf{s} = \mathbf{B}^{-1} \mathbf{W}_{XYZ}$ and scales column c by s_c . This is precisely the **Normalized Primary Matrix** derivation of **SMPTE RP 177** (“the matrix is uniquely determined by the xy chromaticities of the RGB primaries and the XYZ of the white point”) and the method on Bruce Lindbloom's *RGB/XYZ Matrices* page. **Correct.** The unit-test `working_space_white_is_neutral_and_round_trips` confirms $\mathbf{M} \cdot [1, 1, 1]^T = \mathbf{W}_{D65}$ and $\mathbf{M}^{-1} \mathbf{M} = \mathbf{I}$.

1.2.4 2.4 (#4) Row-normalization to avoid double white-balance — color.rs:152-169

Two separate uses must be assessed differently.

(4a) camera_to_working — color.rs:166-169. The pipeline applies white balance *on the mosaic* via `cam_mul` (latent-raw/src/lib.rs:193-207, confirmed called at `main.rs:64` before the matrix at `main.rs:68`). So a neutral patch reaches the matrix as $[\nu, \nu, \nu]$. The code then row-normalizes $\mathbf{M} = \mathbf{M}_{XYZ \rightarrow \text{work}} \cdot \mathbf{M}_{\text{cam} \rightarrow XYZ}$ so each row sums to 1, guaranteeing $[\nu, \nu, \nu] \mapsto [\nu, \nu, \nu]$.

The neutral guarantee is real and the unit test `camera_to_working_keeps_a_neutral_patch_neutral` passes. **But this is not the operation the DNG/dcraw reference pipeline performs, and the two are not colorimetrically equivalent on chromatic colors.** Per Adobe DNG 1.4.0.0 §6 (“Mapping Camera Color Space to CIE XYZ”):

*“CameraToXYZ_D50 = CA * CameraToXYZ ... CA ... is a chromatic adaptation matrix that maps from the white balance xy value to the D50 white point. The recommended method ... is the linear Bradford algorithm.”*

and the preferred (ForwardMatrix) path “causes the white balance adjustment ... to be done by scaling the camera coordinates rather than by adapting the resulting XYZ values.” In both DNG methods the white-balance correction is a **per-input-channel (column) scale on camera coordinates** plus a principled Bradford adaptation. Row-normalization is instead a **per-output-row scale on the working result**. These are *different linear operators*: they coincide only when the camera→XYZ matrix is diagonal, which it never is.

Concretely, with a realistic Canon-EOS-5D-Mark-III `cam_xyz` and its as-shot `cam_mul` (green-normalized $[2.25, 1.0, 1.42]$), I compared the matrix the code applies to white-balanced camera RGB against the matrix a per-channel WB compensation would apply. Both map neutral to neutral, but on saturated inputs they diverge by up to 0.28 in linear working RGB (e.g. a saturated green channel: 1.48 vs 1.20). That is a large, visible hue shift — **not** a sub-quantization effect. The row-normalization therefore *absorbs a real per-illuminant color rotation into a crude three-scalar fudge*. The neutral axis is correct; saturated-color rendering is an uncontrolled approximation whose error grows with the camera matrix's off-diagonality and the WB distance from the matrix's calibration illuminant.

The code comment at `color.rs:164-165` (“the row-normalization stops this matrix from re-applying its own implicit white balance — the classic double-

apply bug”) is misleading: the real fix for double-WB is to build the matrix to *expect* WB’d input (i.e. fold $\text{diag}(\text{cam_mul})^{-1}$ into the matrix, or use the DNG ForwardMatrix path). Row-normalization happens to fix the *neutral* but at the cost of *chromatic* fidelity.

(4b) linear_working_to_linear_srgb — color.rs:152-156. Here both spaces are genuine D65 RGB spaces, so the product $\mathbf{M}_{XYZ \rightarrow sRGB} \cdot \mathbf{M}_{\text{work} \rightarrow XYZ}$ *should* already have rows summing to 1; it does not only because the sRGB matrix is the rounded 4-decimal constant (finding #1). Measured raw row sums: [0.99984, 1.00010, 1.00007], i.e. a neutral tint of at most 1.6×10^{-4} (≈ 0.04 of an 8-bit code, ≈ 10 of a 16-bit code). Row-normalizing removes that tint and shifts chromatic colors by the *same order* — measured **max** 3.25×10^{-4} on saturated red/orange (≈ 0.083 of an 8-bit code, ≈ 21 of a 16-bit code).

The docstring’s “ $\sim 1\text{e-}4$ ” understates the chromatic shift by roughly $3\times$ (actual 3.25×10^{-4}); and note the operation trades a ≤ 10 -LSB neutral tint for a ≤ 21 -LSB chromatic shift at 16-bit — it makes saturated colors move *more* than the error it fixes. Both are sub-perceptual, so the verdict is *correct-with-caveats*: the right underlying fix is to use the 7-decimal sRGB matrix (finding #1), after which no row-normalization here would be needed at all.

1.2.5 2.5 (#5) LUMA_WEIGHTS — color.rs:181

LUMA_WEIGHTS = [0.27881965, 0.72106725, 0.000113055].

Re-deriving the ProPhoto-primaries-at-D65 RGB→XYZ matrix gives a Y row of [0.27881967, 0.72106727, 0.00011305] — agreement to 2×10^{-8} . **The constant is exactly the Y row of the working space**, sums to 1.0, and the unit test luma_weights_match_the_working_matrix guards it. Correct as *relative luminance of the working space*.

Caveat (perceptual operations). The near-zero blue weight (1.1×10^{-4}) is colorimetrically correct for these primaries but has a strong consequence: a fully desaturated pure-blue maps to near-black (luminance ≈ 0.0001 vs. ≈ 0.0722 under Rec. 709/sRGB primaries). The code’s own comment acknowledges this. For a *colorimetric* luminance that is the right answer. But this same luminance() and these same weights feed **perceptual** operations — saturation, clarity, denoise luma (the GPU shader hard-codes the identical triple, color.rs:173-175). For perceptual work the wide-gamut luma is a poor proxy: it makes blue detail nearly weightless, so denoise/clarity will under-protect blue texture and saturation math will darken blues aggressively. This is a *defensible-but-debatable* design choice; many pipelines compute perceptual luma in a display-referred space (Rec. 709 $Y = 0.2126R + 0.7152G + 0.0722B$) precisely to avoid the vanishing-blue artifact. Flagged as a **Medium** note, not an error: it is self-consistent and intentional, but worth a deliberate decision rather than an inherited side effect of the working-space choice.

1.2.6 2.6 (#6) sRGB OETF — latent-export/src/lib.rs:14-29

$$V_{\text{sRGB}} = \begin{cases} 12.92 c & c \leq 0.0031308 \\ 1.055 c^{1/2.4} - 0.055 & c > 0.0031308 \end{cases} \quad c = \begin{cases} V/12.92 & V \leq 0.04045 \\ \left(\frac{V+0.055}{1.055}\right)^{2.4} & V > 0.04045 \end{cases}$$

Every constant — slope **12.92**, encode threshold **0.0031308**, gain **1.055**, exponent **1/2.4**, offset **0.055**, decode threshold **0.04045** — matches IEC 61966-2-1 verbatim (the bg-sRGB amendment text reproduces the identical piecewise form). **Correct.** Endpoints fixed ($0 \mapsto 0$, $1 \mapsto 1$) and the round-trip test passes to 10^{-5} . (Pedantic note: the canonical threshold that makes the two segments meet is 0.0031308 as written; the slightly different 0.04045 on the decode side is the standard’s published companion value — consistent with IEC, not a bug.)

1.2.7 2.7 (#7) highlight_rolloff — latent-export/src/lib.rs:42-51

$$f(x) = \begin{cases} x & x \leq 0.98 \\ 0.98 + 0.02 \cdot \frac{x - 0.98}{(x - 0.98) + 0.02} & x > 0.98 \end{cases}$$

Above the knee this is a **Reinhard compressor** ($t \mapsto t/(t+s)$) rescaled into $[0.98, 1.0)$: monotonic, asymptotic to 1.0, never reaching it. Verified mapping: $f(1.0) = 0.990 \Rightarrow$ sRGB 0.9956 $\Rightarrow$ **254/255** (8-bit), 65246/65535 (16-bit); headroom $x \geq 1.5$ all reach **255** at 8-bit. So the headroom gradient is essentially nonexistent at 8-bit (it lives in the single top code) but real at 16-bit (spanning $\approx 65246 \rightarrow 65535$, ≈ 289 codes).

Assessment: sacrificing exact white (255 $\rightarrow$ 254) to keep a smooth, non-clipping highlight rolloff is a standard **display-referred** trade-off. It is the same dilemma Hable/Filmic tone-mappers describe (“the range 3.25 $\rightarrow$ 4.0 going 254 $\rightarrow$ 255 is not useful”) and solve with overshoot parameters. The code’s choice — anchor faithful $[0, 1]$ to within one code, keep a true gradient above 1.0 for the 16-bit path — is principled and well-documented in-code. **Correct by design**, with the honest limitation (stated in the code) that the 8-bit headroom gradient is degenerate. One genuine subtlety: because the rolloff is applied *per channel independently* (to_display, lib.rs:56-59), a color with one channel above 1.0 and others below will have *only* the hot channel compressed, which **shifts its hue toward the secondaries** in the extreme highlights. A luminance-based or max-channel rolloff would preserve hue better. Low severity (only affects near-clipped highlights), but worth noting.

1.2.8 2.8 (#8) DNG direction convention — color.rs:82-84, latent-raw/src/lib.rs:425-428

camera_to_xyz(xyz_to_cam) = xyz_to_cam.inverse(). Adobe DNG 1.4.0.0 \$Tag-ColorMatrix1: “ColorMatrix1 defines a transformation matrix that converts XYZ values to reference camera native color space values.” §6: XYZtoCamera = AB·CC·CM and (no ForwardMatrix) CameraToXYZ = Inverse(XYZtoCamera). LibRaw’s cam_xyz is the same XYZ $\rightarrow$ camera matrix (LibRaw docs: “cam_xyz should exactly match ColorMatrix2”). The code treats cam_xyz as XYZ $\rightarrow$ camera and inverts it to get camera $\rightarrow$ XYZ — **exactly correct** in direction. Using only the top 3 rows of the $4 \times N$

cam_xyz (lib.rs:427) is fine for the RGBG Bayer case the decoder restricts itself to (is_rgb_bayer, lib.rs:435-436).

1.2.9 2.9 (#9) Chromatic adaptation — omitted

No Bradford adaptation appears anywhere. Given the all-D65 model (finding #2) this is **internally correct**: every stage shares the white point, so an explicit CA would be the identity. The caveat is finding #4a: the *white-balance* correction (which in the DNG model is entwined with a Bradford adaptation from the capture illuminant to the reference white) is handled instead by mosaic cam_mul + row-normalization, which is not equivalent to the DNG Bradford path for saturated colors. So adaptation is correctly omitted *as a white-point transform*, but the *illuminant-dependent color rotation* that DNG bundles into CA is only crudely approximated.

1.3 3. Findings

1.3.1 F-1 — Row-normalizing camera_to_working is not colorimetrically equivalent to the reference WB+adaptation pipeline · High

- **Where:** latent-image/src/color.rs:166-169 (camera_to_working); rationale comment at color.rs:158-165; called from latent-raw/src/lib.rs:425-428, applied at latent-app/src/main.rs:64-70.
- **Issue:** White balance is applied per-channel on the mosaic (cam_mul), then the camera→working matrix is row-normalized to keep neutrals neutral. Row-normalization is a per-output-row scale; the correct WB compensation (DNG ForwardMatrix path, or inverse-ColorMatrix + Bradford CA) is a per-input-channel scale on camera coordinates plus a principled adaptation. The two agree on the neutral axis by construction but diverge on saturated colors — measured up to **0.28 in linear working RGB** (a clearly visible hue/sat shift) for a realistic Canon cam_xyz + as-shot cam_mul. The error scales with the camera matrix’s off-diagonality and the WB distance from the matrix’s calibration illuminant, so it is camera- and white-balance-dependent, not a fixed offset.
- **Reference:** Adobe DNG 1.4.0.0, Ch. 6 “Mapping Camera Color Space to CIE XYZ Space” (CameraToXYZ_D50 = CA·CameraToXYZ, Bradford CA; ForwardMatrix WB by camera-coordinate scaling). LibRaw color-pipeline docs.
- **Recommendation:** Fold the inverse of the applied cam_mul into the matrix instead of row-normalizing — i.e. build $\mathbf{M}_{XYZ \rightarrow \text{work}} \cdot \mathbf{M}_{\text{cam} \rightarrow XYZ} \cdot \text{diag}(\text{cam_mul})^{-1}$ — so WB is mathematically applied exactly once and chromatic colors follow the true camera transform. This still keeps neutrals neutral (when cam_mul is the as-shot neutral) but renders saturated colors correctly. If a strict neutral guarantee under arbitrary user WB is also wanted, adopt the DNG model (Bradford CA from the user WB to D65). At minimum, correct the comment at color.rs:164-165, which mischaracterizes row-normalization as *the* fix for double-WB.

1.3.2 F-2 — Working space is ProPhoto primaries at D65, a non-standard space (not ROMM/ProPhoto) · Medium

- **Where:** `latent-image/src/color.rs:125-136` (`PROPHOTO_PRIMARIES + D65_WHITE, linear_working_to_xyz`).
- **Issue:** ISO 22028-2 fixes ROMM RGB’s reference white at **D50** ($x=0.3457$, $y=0.3585$). The code reuses ROMM’s primaries but at **D65**, producing a space with no standard identity. This is a deliberate, documented choice that keeps the pipeline adaptation-free and is *colorimetrically valid* (the primaries are real and the matrix round-trips), but it must not be confused with ProPhoto RGB; anything tagged “ProPhoto” elsewhere would be a different space.
- **Reference:** ISO 22028-2:2013 / ROMM RGB registry spec (“reference white ... D50”).
- **Recommendation:** Keep the design if the adaptation-free property is wanted, but rename the space in code/comments to something unambiguous (e.g. “ROMM-primaries-D65” / “wide-D65”) so no reader assumes ProPhoto interoperability. No numeric change needed.

1.3.3 F-3 — 4-decimal sRGB matrix leaks precision on the 16-bit export path · Low

- **Where:** `latent-image/src/color.rs:89-93` (`XYZ_TO_LINEAR_SRGB`); consumed by `latent-export/src/lib.rs:95-106` (`save_16`).
- **Issue:** The constant is the IEC *8-bit-grade* 4-decimal matrix. It differs from the full-precision (and the IEC-2003 7-decimal) matrix by up to 3.7×10^{-4} (≈ 24 of a 16-bit code) and does not map D65 white exactly to neutral ($\mathbf{M} \cdot \mathbf{W}_{D65} = [0.99984, 1.0001, 1.00007]$). For an app that exports true 16-bit, this is an avoidable sub-perceptual error; it is also the *root cause* of the working→sRGB row-normalization (F-4).
- **Reference:** IEC 61966-2-1:1999 (4-decimal matrix, “accurate for 8-bit”); Amendment 1:2003 (7-decimal matrix for 16-bit).
- **Recommendation:** Replace with the 7-decimal IEC-2003 matrix, or (cleanest) derive the sRGB matrix from the Rec. 709 primaries with the same `rgb_to_xyz` used for the working space, so both halves of the output transform are full-precision and consistent. The working→sRGB product would then have unit row sums natively and F-4’s row-normalization becomes unnecessary.

1.3.4 F-4 — working→sRGB row-normalization; docstring drift figure understated · Low

- **Where:** `latent-image/src/color.rs:152-156` (`linear_working_to_linear_srgb`), docstring at `color.rs:147-151`.
- **Issue:** Row-normalization here masks the rounded-matrix tint (F-3) rather than fixing it. The docstring claims chromatic colors “shift by the same $\sim 1e-4$,” but the measured maximum is 3.25×10^{-4} (saturated red/orange) — about 3× larger, and *larger than* the 1.6×10^{-4} neutral tint it removes. All sub-perceptual (≤ 0.08 of an 8-bit code; ≤ 21 of a 16-bit code), so impact is negligible, but the comment is inaccurate.

- **Reference:** numerical (re-derived); IEC matrix precision (F-3).
- **Recommendation:** Fix F-3 and delete this row-normalization; otherwise correct the docstring figure to $\sim 3 \times 10^{-4}$.

1.3.5 F-5 — Wide-gamut luma drives perceptual operations (saturation/clarity/denoise) · Medium

- **Where:** latent-image/src/color.rs:181-186 (LUMA_WEIGHTS, luminance); duplicated in the GPU shader (color.rs:173-175 note).
- **Issue:** The weights are correct *colorimetric* luminance for the working space, but the blue weight is $\approx 1.1 \times 10^{-4}$ (vs 0.0722 in Rec. 709). Using this luma for perceptual ops makes blue carry almost no weight: denoise/clarity under-protect blue texture and saturation/value math darkens blues hard (a fully-desaturated pure blue $\rightarrow$ near-black). Self-consistent and intentional, but a debatable choice.
- **Reference:** Rec. 709/sRGB luma coefficients $Y = 0.2126R + 0.7152G + 0.0722B$ (IEC 61966-2-1 / ITU-R BT.709); common practice of computing perceptual luma in a display-referred space.
- **Recommendation:** Decide per-operation. For exposure/relative-luminance, keep the working luma. For *perceptual* operations (clarity, denoise weighting, saturation), consider a Rec. 709 luma (computed after a notional working $\rightarrow$ sRGB primaries rotation) so blue detail isn't treated as nearly weightless. At minimum document that perceptual ops inherit the vanishing-blue behavior deliberately.

1.3.6 F-6 — Per-channel highlight rolloff can shift hue in extreme highlights · Low

- **Where:** latent-export/src/lib.rs:56-59 (to_display applies highlight_rolloff independently per channel).
- **Issue:** When one channel exceeds 1.0 and others don't, only the hot channel is compressed, pulling near-clipped highlight colors toward a secondary/white in a hue-shifting way. Only affects pixels with headroom in some channels.
- **Reference:** display-referred tone-mapping practice (luminance- or max-channel-based rolloff preserves hue).
- **Recommendation:** Consider rolling off on the max channel (or luminance) and scaling the triplet, rather than per-channel, if highlight hue fidelity matters. Low priority.

1.3.7 F-7 — Note: matrix algebra, OETF, DNG direction, and rgb_to_xyz are all correct · Note

- Mat3 inverse/det/mul, the SMPTE-RP-177 rgb_to_xyz construction, the sRGB OETF constants, the DNG XYZ $\rightarrow$ camera direction, and the round-trip/neutral unit tests are all correct and well-tested. The ICC embedding uses a real validated moxcms sRGB profile that matches the output transform. These are called out explicitly as **correct** so the findings above are read as targeted, not a blanket concern.

Severity tally: Critical 0 · High 1 · Medium 2 · Low 3 · Note 1.

1.4 4. References

1. **IEC 61966-2-1:1999**, *Multimedia systems and equipment — Colour measurement and management — Part 2-1: Colour management — Default RGB colour space — sRGB*. (Paywalled; cited for the canonical sRGB matrix, OETF constants 12.92 / 0.0031308 / 1.055 / 1/2.4 / 0.055, and Rec. 709 primaries + D65 white.)
2. **IEC 61966-2-1:1999/Amd.1:2003** — adds the 7-decimal XYZ→sRGB matrix for 16-bit. Freely available specification text reproduced as “*Specification of bg-sRGB (Amendment 1 to IEC 61966-2-1)*”, color.org. <https://www.color.org/bgsrgb.pdf> — saved: docs/color-srgb-iec61966-2-1-amd1-bgsrgb.pdf
3. **ISO 22028-2:2013**, *Photography and graphic technology — Extended colour encodings ... Part 2: ROMM RGB*. Specification text (primaries, **D50** reference white, RGB→XYZ matrix) from the ICC RGB registry. <https://registry.color.org/rgb-registry/files/ROMMRGB.pdf> — saved: docs/color-romm-rgb-iso22028-2.pdf
4. **Adobe, Digital Negative (DNG) Specification, Version 1.4.0.0** (2012). \$Tags ColorMatrix1/2 (“converts XYZ values to reference camera native color space values”); Ch. 6 “*Mapping Camera Color Space to CIE XYZ Space*” (XYZtoCamera = AB·CC·CM, CameraToXYZ = Inverse(XYZtoCamera), CameraToXYZ_D50 = CA·CameraToXYZ with linear Bradford CA; ForwardMatrix WB by camera-coordinate scaling). https://www.kronometric.org/phot/processing/DNG/dng_spec_1.4.0.0.pdf — saved: docs/color-dng-spec-1.4.0.0.pdf
5. **SMPTE RP 177:1993 (R2002)**, *Derivation of Basic Television Color Equations*. Defines the Normalized Primary Matrix (primaries-as-columns scaled to the white point) used by rgb_to_xyz. (Paywalled standard; cited for the construction method.) <https://standards.globalspec.com/std/1284890/smp-te-rp-177>
6. **Bruce Justin Lindbloom**, *RGB/XYZ Matrices and RGB Working Space Information*. Reference ProPhoto-D50 RGB→XYZ matrix (matched to 2×10^{-5}) and the same NPM derivation. http://www.brucelindbloom.com/index.html?Eqn_RGB_XYZ_Matrix.html
7. **ITU-R BT.709-6**, *Parameter values for the HDTV standards ...* — Rec. 709 primaries and luma coefficients 0.2126/0.7152/0.0722, the display-referred perceptual-luma reference for finding F-5.
8. **mina86**, *Calculating RGB→XYZ matrix* (2019) — independent derivation cross-check of the NPM construction. <https://mina86.com/2019/srgb-xyz-matrix/>
9. **J. Hable / “Filmic Worlds”**, *Filmic Tonemapping with Piecewise Power Curves* — the 254→255 highlight-collapse trade-off and overshoot technique, context for finding #7 / F-6. <https://filmicworlds.com/blog/filmic-tonemapping-with-piecewise-power-curves/>
10. **E. Reinhard et al.**, *Photographic Tone Reproduction for Digital Images* (SIGGRAPH 2002) — the $L/(L+1)$ compressor that highlight_rolloff rescales.

Downloaded reference PDFs (freely available, authoritative): docs/color-srgb-iec61966-2-1-amd1-bgsrgb.pdf, docs/color-romm-rgb-iso22028-2.pdf, docs/color-dng-spec-1.4.0.0.pdf. Paywalled standards (IEC 61966-2-1 base text, ISO 22028-2 full text, SMPTE RP 177) are cited but not redistributed.

Chapter 2

Audit 02 — RAW Decode & Demosaic

Component: latent-raw **Files audited:** latent-raw/src/lib.rs, latent-raw/build.rs (with cross-reference to latent-image/src/color.rs) **Scope:** RAW unpack via LibRaw (unpack-only), black/white normalization, per-CFA white balance on the mosaic, bilinear and Malvar-He-Cutler (MHC) demosaic, highlight reconstruction, CFA handling, and the camera→working color matrix entry point. **Date:** 2026-06-27 **Method:** Code read line-by-line, then every numeric/algorithmic claim verified against primary sources (Malvar-He-Cutler ICASSP 2004 paper Fig. 2 read element-by-element from the rendered PDF, the IPOL reproduction, the Adobe DNG Specification 1.6.0.0 Chapter 5 and tag definitions, and the LibRaw API/forum documentation). PDFs cited are downloaded under docs/.

2.1 1. Summary of the Decode / Demosaic Model

latent uses LibRaw purely as an **unpacker** (libraw_open_file → libraw_unpack), reading the raw Bayer mosaic (rawdata.raw_image, u16) and metadata (color, idata, other, lens), then runs its **own** pipeline. The model, in order:

1. **Normalize** (RawImage::normalized): per photosite, look up its CFA color $c \in \{0, 1, 2, 3\}$ from the 2×2 cfa[] pattern and map

$$\tilde{s} = \max\left(0, \frac{s - (\text{black} + \text{cblack}[c])}{\max(1, \text{white} - (\text{black} + \text{cblack}[c]))}\right),$$

where $\text{black} = \text{color.black}$, $\text{cblack}[c] = \text{color.cblack}[c]$, and $\text{white} = \text{color.maximum}$. Values above 1.0 are **kept** (highlight headroom); only the floor is clamped.

2. **Clip mask** (clip_mask): boolean per photosite, raw_sample as u32 $\geq \text{color.maximum}$, computed on the raw integers.
3. **White balance on the mosaic** (apply_white_balance): cam_mul normalized so green = 1, applied per CFA color **before** demosaic.

4. **Demosaic**: demosaic_bilinear (3×3 same-color averaging) or demosaic_mhc (5×5 MHC interior, 2-pixel bilinear border).
5. **Highlight reconstruction** (reconstruct_highlights): post-demosaic, in white-balanced camera RGB; where ≥ 2 channels are flagged clipped (via the exact raw mask propagated through the 3×3 neighborhood), rebuild the clipped channels up to the pixel's brightest channel, keeping measured channels.
6. **Color matrix** (color_matrix $\rightarrow$ color::camera_to_working): invert cam_xyz (XYZ $\rightarrow$ camera), compose with XYZ $\rightarrow$ linear-ProPhoto-D65, row-normalize so neutral stays neutral.

Working space is linear ProPhoto primaries pinned to D65; the pipeline is linear-light f32.

The data path is sound and the matrix/WB plumbing avoids the classic double-white-balance bug (via row-normalization). The MHC kernels are transcribed **correctly**. The principal correctness gaps are in **level metadata**: the code ignores the 2-D cblack black-level *pattern* (cblack[4..]) and the per-channel white level (color.linear_max[]), and it uses a per-channel rescale denominator that differs from the DNG model. It also accepts X-Trans files that its 2×2 Bayer demosaic cannot handle.

2.2 2. Point-by-Point Verification

2.2.1 2.1 Black-level handling — cblack pattern ignored

Code (lib.rs:164-165):

```
let color = self.meta.cfa[(i / w % 2) * 2 + (i % w % 2)] as usize;
let black = base + self.meta.cblack[color] as f32; // base = color.black
```

read_metadata (lib.rs:521-522) copies only color.black and color.cblack[0..4].

Authoritative layout. LibRaw's libraw_colordata_t.cblack[] is documented as: *“Per-channel black level correction. First 4 values are per-channel correction, next two are black level pattern block size, then cblack[4]*cblack[5] correction values (for indexes [6 ... 6+cblack[4]*cblack[5]).”* (LibRaw, *Data Structures and Constants*.) Equivalently, the per-pixel black level is

$$\text{black}_{\text{pixel}} = \text{black} + \text{cblack}[c] + \text{cblack}[6 + (r \bmod H_b) W_b + (k \bmod W_b)],$$

with $W_b = \text{cblack}[4]$, $H_b = \text{cblack}[5]$. This mirrors the DNG model exactly: the BlackLevel tag is *“the zero light ... encoding level, as a repeating pattern,”* sized by BlackLevelRepeatDim (BlackLevelRepeatRows $\times$ BlackLevelRepeatCols), origin top-left of ActiveArea, stored row-column-sample (DNG 1.6.0.0, Chapter 5 and the BlackLevel/BlackLevelRepeatDim tag definitions). The DNG black level per pixel is the **sum** BlackLevel + BlackLevelDeltaH + BlackLevelDeltaV.

The code reads cblack[0..4] only and **never** consults cblack[4], cblack[5], or the pattern values at cblack[6..]. For bodies that deliver their pedestal via the pattern (LibRaw's confirmed example: **Fujifilm X-Trans stores ALL black in the pattern**,

cblack[4]==cblack[5]==6, with cblack[0..3] and even black possibly 0; many CFA bodies populate a 2×2 pattern), the subtracted black is wrong — typically too small, leaving **raised, tinted shadows** and a non-zero black floor. Note cblack[4] is *reinterpreted* by this code as “the G2 channel’s per-channel black,” which is semantically wrong: cblack[4] is the pattern **width**, not a 5th channel. (The code’s cblack array is [u32;4], so it cannot even hold index 4 — it is structurally incapable of reading the pattern.)

Note on LibRaw’s own normalization: LibRaw’s `subtract_black()/raw2image` *does* fold the pattern into a per-pixel black before scaling. Because this code is unpack-only and re-implements normalization, it must replicate that folding itself.

VERDICT: Incorrect (per-channel scalar path) / Incomplete (pattern path).

The scalar `black + cblack[c]` term is correct as far as it goes, but omitting the `cblack[4..]` 2-D pattern mishandles every camera that encodes black there (notably X-Trans, and a number of Bayer bodies). **Citations:** LibRaw *Data Structures and Constants* (cblack layout); LibRaw issue #107 (X-Trans black detected as 0; pattern usage); DNG 1.6.0.0 §“Black Subtraction” + `BlackLevel/BlackLevelRepeatDim` tags.

2.2.2 2.2 White level — scalar `color.maximum`, per-channel `linear_max` ignored

Code (lib.rs:181-186, 522): `clip_mask` and `normalized` both use the single scalar `white = color.maximum`.

Authoritative semantics. LibRaw exposes three distinct fields (LibRaw *Data Structures and Constants*; forum *Maximum/saturation pixel value, color.maximum and camera white level*): - `color.maximum` — “Maximum pixel value. Calculated from the data for most cameras, hardcoded for others ... may be changed on post-processing ... and by automated maximum adjustment.” It is a **format/guessed** saturation, available right after `unpack()`. It can be optimistic or conservative. - `color.linear_max[4]` — “Per-channel linear data maximum read from file metadata ... set to zero” if absent. This is the **vendor “specular white”** saturation, **per channel**. (Documented example: Nikon D70s `maximum = 4095`, `linear_max = {3827,3827,3827,3827}` — a ~7% difference.) - `color.data_maximum` — actual per-file max, only filled at `raw2image/dcraw_process`, i.e. **not** available in this unpack-only flow.

Why this matters (magenta highlights). At the sensor all channels clip at (approximately) the same level, but per-channel saturation can differ, and the white-balance gains differ per channel. Using a *single* maximum that is **too high** lets a channel that actually saturated slip below the clip threshold, so it is treated as valid and not reconstructed; after per-channel WB the channels land at different heights and a blown neutral renders **pink/magenta**. The LibRaw author’s explicit recommendation for exactly this “pink clouds” failure is to “**use camera-provided `linear_max` values (if any)**” and to derive the saturation from a robust histogram that ignores hot pixels, rather than trusting a single inflated maximum (LibRaw forum *Pink Clouds with LibRaw*

0.20.0).

The DNG model is also explicitly **per-channel**: the WhiteLevel tag has Count = SamplesPerPixel, i.e. one white level per CFA plane.

VERDICT: Questionable. A single scalar maximum is the easy choice and is *adequate when maximum is accurate and channels share a saturation*, but it ignores LibRaw’s own recommended linear_max[4] (which is per-channel and available after unpack()), and it can directly cause magenta highlight casts when maximum is miscalibrated. Both the clip mask and the normalization denominator should prefer per-channel linear_max[c] when non-zero. **Citations:** LibRaw *Data Structures and Constants*; LibRaw forum nodes 2238, 2473, 2605 (pink clouds); DNG 1.6.0.0 WhiteLevel tag (Count = SamplesPerPixel).

2.2.3 2.3 Normalization formula vs. DNG linearization

Code (lib.rs:166-170): scale = 1 / max(1, white - black_c), then (s - black_c) * scale floored at 0; ceiling **not** clamped (headroom kept). Per channel, black_c = black + cblack[c].

DNG model (1.6.0.0, Chapter 5, “Mapping Raw Values to Linear Reference Values”): Linearization (LUT) → Black Subtraction (BlackLevel + DeltaH + DeltaV) → **Rescaling** with “scale factor ... the inverse of the difference between the value specified in the WhiteLevel tag and the **maximum computed black level for the sample plane**” → Clipping to [0, 1].

Two deviations: 1. **Per-channel denominator vs. DNG’s single per-plane denominator.** The code rescales each channel by its *own* (white – black_c). The DNG formula uses one denominator per sample plane built from the **maximum** black level. With different cblack[c] per channel, the code gives each channel a *slightly different gain* even before white balance: a red site with cblack=+200 is scaled by 1/(W – B – 200) while green is scaled by 1/(W – B). For a saturated photosite this is *intentional and arguably desirable* (the code’s own test normalize_subtracts_per_channel_black shows every channel’s white lands at exactly 1.0), but for **mid-tones it introduces a small per-channel gain difference** (a faint tint) that the subsequent cam_mul white balance does not exactly cancel, because WB multiplies by a constant gain, not by a per-channel range ratio. In practice cblack[c] spreads are tiny (tens of DN out of thousands), so the tint is sub-LSB at 8-bit; it is a real but negligible deviation, *as long as cblack spreads stay small*. It would matter more for sensors with large per-channel black offsets. 2. **No upper clip.** Keeping values > 1 is a deliberate, defensible choice for a floating-point pipeline with separate highlight handling (and is *more* information-preserving than DNG’s clip-to-1). This is correct for the architecture; saturation is tracked by clip_mask instead. Fine. 3. **Black subtraction vs. linearization order:** the code has no LinearizationTable step. LibRaw applies the linearization curve during unpack(), so raw_image is already linearized — this is correct and not a gap.

VERDICT: Correct-with-caveats. The structure (per-channel black subtract → rescale → floor, headroom preserved) matches standard raw linearization. The per-channel *denominator* diverges from DNG’s single per-plane scale and can introduce

a slight mid-tone tint when per-channel black offsets are large; negligible for typical bodies. **Citation:** DNG 1.6.0.0, Chapter 5 (Black Subtraction / Rescaling / Clipping).

2.2.4 2.4 White balance on the mosaic, before demosaic

Code (lib.rs:193-207): `cam_mul` normalized to green = 1 (with a G2-zero fallback to G1), applied per CFA color **before** demosaic. Gains `[m0/g, 1, m2/g, g2/g]`.

Standard order. Applying as-shot WB multipliers to the mosaic *before* demosaic is standard and is exactly what **darktable** does: “*the white balance as reported by the camera is used in processing modules before input color profile ... before ... demosaic ... so that demosaic works correctly*” (darktable manual, white-balance module). dcrw’s `scale_colors()` likewise pre-multiplies the mosaic before interpolation. (RawTherapee computes *auto*-WB statistics after demosaic, but the as-shot multipliers are still applied to the mosaic; the post-demosaic step there is an analysis pass, not where the camera multipliers are applied.)

Does WB-before-MHC violate MHC’s assumptions? MHC’s gradient correction assumes inter-channel correlation (a sharp luminance edge appears in all channels). A **constant per-channel scalar** gain preserves edge *positions* and the proportionality the Laplacian correction exploits — it only rescales amplitudes. Because the correction term for, e.g., G-at-R is $\alpha \cdot \Delta_R$ where Δ_R is a Laplacian of the **same** (red) channel that is being WB-scaled together with the green bilinear estimate, a uniform per-channel scale does **not** break the design (the well-known caveat is the opposite ordering issue — clipping/non-linearity before demosaic, not linear WB). darktable’s choice to WB before demosaic confirms this is accepted practice and, if anything, improves highlight/CA behavior. One genuine subtlety: WB pushes some channels well above 1.0, so the linear MHC filter can produce overshoot near clipped edges — but that is inherent to linear demosaic and handled downstream by highlight reconstruction.

VERDICT: Correct. WB on the mosaic before demosaic is the standard order; a linear per-channel gain does not violate MHC’s gradient assumptions. **Citations:** darktable manual (white balance / demosaic ordering); dcrw `scale_colors`.

2.2.5 2.5 Malvar-He-Cutler 5×5 kernels — element-by-element

Verified against **Malvar, He & Cutler, ICASSP 2004, Fig. 2** (read directly from the rendered PDF at 400 DPI) and cross-checked against the IPOL reproduction. All filters are divided by 8 (the paper states coefficients are scaled by 8; gains $\alpha = 1/2$, $\beta = 5/8$, $\gamma = 3/4$).

(a) G at R/B locations — code `G_AT_RB` (lib.rs:274-280). Paper “G at R locations” /

“G at B locations” cross:

$$\frac{1}{8} \begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ -1 & 2 & 4 & 2 & -1 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{bmatrix}$$

Center 4, axis-1 neighbors 2, axis-2 neighbors -1. **Matches.**

(b) R/B at green, same-row case — code ROW (lib.rs:291-297). Paper “R at green in R row, B column” (and symmetric “B at green in B row, R column”):

$$\frac{1}{8} \begin{bmatrix} 0 & 0 & \frac{1}{2} & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & 4 & 5 & 4 & -1 \\ 0 & -1 & 0 & -1 & 0 \\ 0 & 0 & \frac{1}{2} & 0 & 0 \end{bmatrix}$$

Center 5, horizontal-1 = +4, four diagonal-adjacent = -1, horizontal-2 = -1, vertical-2 = + $\frac{1}{2}$. **Matches** (confirmed against IPOL: center +5, ± 1 horiz +4, diagonals -1, horiz dist-2 -1, vert dist-2 +1/2).

(c) R/B at green, same-column case — code COL (lib.rs:300-306). Paper “R at green in B row, R column” (transpose of (b)):

$$\frac{1}{8} \begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 \\ \frac{1}{2} & 0 & 5 & 0 & \frac{1}{2} \\ 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{bmatrix}$$

Matches — exact transpose of ROW, as the code comment claims.

(d) R at B / B at R (diagonal / checkerboard) — code DIAG (lib.rs:282-288). Paper “R at blue in B row, B column” (and symmetric “B at red”):

$$\frac{1}{8} \begin{bmatrix} 0 & 0 & -\frac{3}{2} & 0 & 0 \\ 0 & 2 & 0 & 2 & 0 \\ -\frac{3}{2} & 0 & 6 & 0 & -\frac{3}{2} \\ 0 & 2 & 0 & 2 & 0 \\ 0 & 0 & -\frac{3}{2} & 0 & 0 \end{bmatrix}$$

Center 6, four diagonal-1 = +2, axis-2 = - $\frac{3}{2}$. Code uses $-1.5 = -\frac{3}{2}$. **Matches.**

Assignment logic (lib.rs:319-342). center_ch comes from channel_at. For R sites: $G = G_AT_RB$, $B = DIAG$. For B sites: $G = G_AT_RB$, $R = DIAG$. For green sites the code inspects the **right** neighbor: if channel_at(x+1,y)==0 (red is the horizontal neighbor, i.e. the green sits in a red row), then R uses ROW and B uses COL; else swapped. This is exactly the paper’s “R at green in **R row**” → ROW vs. “R at green in **B row**” → COL distinction, applied symmetrically to B. The x+1 read is in-bounds because MHC only runs when x+2 < w. **Correct.**

VERDICT: Correct. Coefficients, the 1/8 normalization, the ROW/COL row-vs-column assignment, and the diagonal R↔B filter all match the source element-by-element. Independently corroborated by the synthetic round-trip test `mhc_beats_bilinear_on_detail` and `all_cfa_phases_reconstruct_equally`. **Citations:** Malvar-He-Cutler 2004 Fig. 2 ([docs/demosaic-malvar-he-cutler-2004.pdf](https://docs.demosaic-malvar-he-cutler-2004.pdf)); IPOL *Malvar-He-Cutler Linear Image Demosaicking*.

2.2.6 2.6 2-pixel MHC border fallback to bilinear

Code (`lib.rs:353-357`): pixels with the full 5×5 window out of bounds (`x<2 || y<2 || x+2>=w || y+2>=h`) fall back to `bilinear_pixel`.

A 5×5 filter genuinely cannot be evaluated within 2 px of the edge without padding. Bilinear there is the standard, conservative choice (`dcraw/RawTherapee` similarly degrade or mirror at borders). Two minor notes: (1) MHC implementations sometimes *mirror/clamp* the window to keep the higher-quality filter to the very edge; bilinear is simpler and a touch softer on a 2-px frame — cosmetically invisible on real images. (2) The bilinear fallback itself handles borders correctly (see 2.9).

VERDICT: Correct-with-caveats (acceptable). A 2-px bilinear frame is a reasonable, common fallback; a mirrored-edge MHC would be marginally sharper but is not required for correctness.

2.2.7 2.7 Highlight reconstruction

Code (`reconstruct_highlights + clipped_channels, lib.rs:369-416`): using the **exact raw clip mask** (not a post-demosaic value threshold), a channel at a pixel is “clipped” if its own photosite saturated (center channel) or any same-color photosite in the 3×3 neighborhood saturated (interpolated channels). Where ≥ 2 channels are clipped, the clipped channels are raised to the pixel’s **brightest** channel (`peak = max(R,G,B)`); measured channels are kept untouched.

Comparison to established methods. `dcraw`’s `-H` ladder is the canonical reference (`dcraw` man page; *LibRaw Understanding highlight modes*): `0=clip` (all to white), `1=unclip` (leave pink), `2=blend`, `3..9=rebuild` (low favors white, high favors color). The `dcraw` `LCH-blend (-G)` sets L and H from the unclipped neighborhood and C from the clipped pixel. `RawTherapee` offers Luminance Recovery (gray fill), CIE Lab blending, and **Color Propagation** (bleeds surrounding known color into the clipped region) (`RawPedia Exposure`; `RawTherapee` issue #3311).

This code’s method is a **single-channel-preserving “rebuild toward neutral peak”** — close in spirit to `dcraw` rebuild modes with a low color-favoring number, with one genuinely good property: by gating on **the exact raw mask** and on ≥ 2 **clipped channels**, it (a) avoids flattening a genuinely saturated single-channel color (a real red light keeps its hue — verified by the test `reconstruct_highlights_rebuilds_blowed_channels_keeping_measured_ones`), and (b)

fixes the dominant artifact, neutral highlights going magenta. That is a sensible, conservative default.

Limitations (all “the more advanced methods do more”): - **No spatial propagation**. It reconstructs each pixel from its own channels only; it cannot recover *texture/gradient* in a fully-blown region the way Color Propagation or guided/LCH-blend methods do. A large blown cloud becomes a flat plateau at peak. - **peak = max channel can be wrong** when the brightest channel is itself clipped at a per-channel-too-low white level (ties into 2.2): if maximum is conservative, peak undershoots and highlights render dim/gray; if optimistic, residual color remains. Using per-channel `linear_max` would make peak more trustworthy. - **The 3×3 mask propagation** for interpolated channels is a reasonable approximation of “this channel was drawn from a saturated same-color sample,” but it does not match the 5×5 MHC support — an MHC-interpolated channel can draw from a clipped sample up to 2 px away that the 3×3 test misses, so a thin ring of MHC pixels just outside the 3×3 reach can keep a faint cast. Minor. - It runs in **white-balanced camera RGB before the color matrix**, which is the correct stage (the clipping geometry is per-CFA there).

VERDICT: Correct-with-caveats. A defensible, conservative reconstruction that correctly targets the magenta-highlight problem and preserves genuine saturated colors; weaker than color-propagation/LCH methods for recovering structure in large blown regions, and its quality is bounded by the white-level accuracy from 2.2. The code’s own comment (“Finer color propagation can come later”) acknowledges this.

Citations: dcraw man page (-H, -G); LibRaw *Understanding highlight modes* (node 2694); RawPedia *Exposure*; dcraw LCH-blend patch.

2.2.8 2.8 CFA handling

`is_rgb_bayer` (lib.rs:435-437): accepts iff `cdesc == b"RGBG"`.

Problem — X-Trans (and any RGB non-Bayer) passes this guard. Per LibRaw (Alex Tutubalin, forum node 2561): “*cdesc is not a pattern, it is just an indication (RGB or CMYK or CMYG, etc.), so for all four R-G-B-G2 orders possible cdesc will be set to RGBG.*” X-Trans sensors are **also R/G/B**, so their `cdesc` is **likewise “RGBG”**. X-Trans is distinguished by `idata.filters == 9` and a 6×6 `idata.xtrans[][]` pattern — **not** by `cdesc`. Consequently a Fujifilm X-Trans .RAF would pass `is_rgb_bayer`, and then be demosaiced by 2×2-Bayer logic that reads `cfa[(y%2)*2 + (x%2)]` — producing a **scrambled, mis-colored** image rather than the intended clean rejection. The decode guard’s stated intent (“reject CYGM/RGBE/X-Trans”) is only met for CYGM/RGBE (whose `cdesc` differs); **X-Trans is not actually rejected**.

`channel_at` (lib.rs:210-213): `c = cfa[(y%2)*2 + (x%2)]`, then `(c==2)*2 + (c==1||c==3)`. Mapping:

CFA index <i>c</i>	meaning	result	RGB channel
0	R	$0 \cdot 2 + 0$	0 (R) ✓
1	G	$0 \cdot 2 + 1$	1 (G) ✓
2	B	$1 \cdot 2 + 0$	2 (B) ✓

CFA index <i>c</i>	meaning	result	RGB channel
3	G2	$0 \cdot 2 + 1$	1 (G) ✓

Correct for all four colors, and because it reads the *actual* `cfa[]` (filled by `libraw_COLOR(raw, row, col)` in `read_metadata`, `lib.rs:512-516`), it is correct for **all four Bayer phases** (RGGB / BGGR / GRBG / GBRG). This is empirically confirmed by `all_cfa_phases_reconstruct_equally` (every phase MAE < 0.03). G2→green is handled correctly everywhere.

VERDICT (channel mapping & phases): Correct. **VERDICT (sensor-type guard): Incorrect** — `is_rgb_bayer` lets X-Trans through; the guard should also require a Bayer filters value (e.g. `filters != 9` and `filters != 0`) and/or `colors == 3` with a 2×2 layout. (`read_metadata` does not currently read `idata.filters`/`idata.colors`.) **Citations:** LibRaw forum node 2561 (cdesc is RGBG for all Bayer orders); LibRaw *Data Structures and Constants* (`filters==9` = X-Trans, `idata.xtrans`); LibRaw node 2301.

2.2.9 2.9 Bilinear demosaic quality

Code (`bilinear_pixel`, `lib.rs:218-246`): `known_channel` = the sample; each missing channel = mean of same-color samples in the 3×3 neighborhood, clamped at borders; a pixel with no same-color neighbor (degenerate tiny image) falls back to the center value.

This is textbook separable bilinear demosaic (the paper’s Eqs. (1) with the cross/diagonal neighbor sets). It is **exact for constant and linear signals** (the round-trip tests `roundtrip_of_constant_image_is_lossless` and `bilinear_reconstructs_a_smooth_gradient_well` confirm MAE 0 and <0.01). Border handling is correct: the `nx<0 || ny<0 || nx>=w || ny>=h` guard simply averages the *available* same-color neighbors, which is the standard “shrink the window at the edge” behavior — no out-of-bounds reads, no wraparound. The only quality limitation is the well-known one (zipper/false-color on high-frequency edges) inherent to channel-independent bilinear, which is precisely why MHC exists and is the default-quality path.

VERDICT: Correct. Standard bilinear; border-safe; exact on smooth content; the expected (acceptable) softness/artifacts on detail.

2.2.10 2.10 FFI / build (`build.rs`, `unpack`)

Not a numeric-correctness item, but in scope. `build.rs` links LibRaw via `pkg-config` (dynamic — LGPL-clean) and allowlists only `libraw_*` symbols. `unpack` follows LibRaw’s `open` → `unpack` lifecycle with checked return codes, a RAII Handle (`libraw_close` on drop on every path), and a null check on `rawdata.raw_image`. Width/height use `sizes.raw_width/raw_height` (the **full** unpacked buffer including

masked/optical-black borders), and the mosaic length is `raw_width*raw_height` — consistent with indexing `raw_image` directly. **Note:** because dimensions are the *raw* (not *visible/active*) area, the CFA phase from `libraw_COLOR(raw,0,0)` and the `cfa[]` lookup are anchored at the raw top-left, which is the correct origin for `raw_image`. No correctness issue spotted here; this matches LibRaw’s documented `raw_image` semantics.

VERDICT: Correct.

2.3 3. Findings by Severity

2.3.1 CRITICAL

C1 — cblack 2-D black-level pattern (cblack[4..]) is ignored. - **Where:** `lib.rs:520-522` (`read_metadata` copies only `cblack[0..4]`; `Metadata.cblack` is `[u32;4]`, while the FFI field is `cblack: [c_uint; 4104]`), consumed at `lib.rs:164-165`. - **Issue:** LibRaw/DNG encode a repeating 2-D black pattern in `cblack[4]=W`, `cblack[5]=H`, values at `cblack[6 .. 6+W*H]`. The code never reads it; index 4 is *mis-used* as a “G2 channel black.” Cameras that store their pedestal in the pattern (confirmed for **Fujifilm X-Trans**, where `cblack[0..3]` and `black` may be 0 and *all* black is in the 6×6 pattern; also various Bayer bodies with 2×2 patterns) get the wrong (usually too small) black subtracted ⇒ raised, color-tinted shadows and a non-zero black floor. - **Reference:** LibRaw *Data Structures and Constants* (`cblack` layout); DNG 1.6.0.0 Ch.5 + `BlackLevel/BlackLevelRepeatDim` tags; LibRaw issue #107. - **Recommendation:** Read the full `color.cblack[]` (the generated binding is `cblack: [c_uint; 4104]` — confirmed in `OUT_DIR/bindings.rs`; the code currently widens only indices 0..4 into its `[u32;4]`). Copy `cblack[0..6]` plus the `W*H` pattern values. Compute per-pixel black as `black + cblack[c] + cblack[6 + (row%H)*W + (col%W)]` (guarding `W==0 || H==0`). At minimum, detect `cblack[4]*cblack[5] > 0` and fold the pattern in; this also unblocks correct X-Trans black if/when X-Trans demosaic is added.

2.3.2 HIGH

H1 — Per-channel white level (color.linear_max[4]) ignored; scalar maximum can cause magenta highlights. - **Where:** `lib.rs:181-186` (`clip_mask`), `lib.rs:166 & 522` (`normalized / read_metadata`). - **Issue:** Both clip detection and normalization use only `color.maximum`, a single format/guessed value that “*may be changed by automated maximum adjustment*” and can be optimistic. LibRaw’s own remedy for pink/magenta blown highlights is to prefer per-channel `linear_max[]`. With a single wrong maximum, a truly-saturated channel can evade the clip mask and, after per-channel WB, render the highlight magenta. - **Reference:** LibRaw *Data Structures and Constants*; LibRaw forum nodes 2238, 2473, **2605 (Pink Clouds)**; DNG 1.6.0.0 `WhiteLevel` (`Count = SamplesPerPixel`). - **Recommendation:** Read `color.linear_max[4]`; where `linear_max[c] != 0`, use it as the per-channel white for **both** the clip mask and the normalization denominator (the clip test

then becomes per-CFA-channel). Keep maximum as the fallback. This also makes `reconstruct_highlights`' peak trustworthy (see N2).

H2 — `is_rgb_bayer` accepts X-Trans, which the 2×2 demosaic cannot handle. - **Where:** `lib.rs:435-437`, gate at `lib.rs:475-477`. - **Issue:** `cdesc == "RGBG"` is true for **every** RGB sensor including **X-Trans** (`cdesc` is a color *indication*, not a pattern). X-Trans is identified by `idata.filters == 9 / idata.xtrans`, which the code never reads. An X-Trans .RAF passes the guard and is then mis-demosaiced as Bayer = scrambled color, contradicting the guard's documented intent. - **Reference:** LibRaw forum node 2561 (`cdesc=RGBG` for all Bayer orders); *LibRaw Data Structures and Constants* (`filters==9` = X-Trans). - **Recommendation:** Read `idata.filters` and `idata.colors`; accept only standard 2×2 Bayer (e.g. `colors == 3 && filters != 0 && filters != 9`). Reject `filters == 9` (X-Trans), `filters == 0` (non-Bayer/Fovean/linear), and ≠3-color CFAs explicitly.

2.3.3 MEDIUM

M1 — Per-channel rescale denominator deviates from the DNG single-plane scale. - **Where:** `lib.rs:166-169`. - **Issue:** Normalizing each channel by its own (`white - black_c`) (rather than DNG's single `1/(WhiteLevel - max_black)` per plane) gives channels slightly different gains; for mid-tones this is a faint tint that WB does not exactly cancel. Negligible for small `cblack` spreads, larger if a body has big per-channel black offsets. - **Reference:** DNG 1.6.0.0 Ch.5 ("Rescaling": inverse of `WhiteLevel - maximum computed black level for the sample plane`). - ****Recommendation:**** Either adopt DNG's per-plane scale (`white - max_cblack_c`) for all channels, or — if per-channel white is implemented (H1) — use(`linear_max[c] - black_c`)' consistently so the per-channel normalization is colorimetrically intentional rather than a side effect of per-channel black.

2.3.4 LOW

L1 — MHC highlight-clip mask uses 3×3 propagation but MHC support is 5×5. - **Where:** `clipped_channels` (`lib.rs:374-386`) vs. `mhc_pixel` 5×5 support. - **Issue:** An MHC-interpolated channel can draw from a saturated same-color sample 2 px away that the 3×3 clip-propagation misses, leaving a faint cast on a thin ring of MHC pixels at the edge of blown regions. - **Recommendation:** For the MHC path, propagate the clip flag over the same 5×5 same-color support (or simply over a 5×5 window); cheap and tightens highlight recovery.

L2 — MHC border could mirror/clamp instead of dropping to bilinear. - **Where:** `lib.rs:353-357`. A 2-px bilinear frame is slightly softer than mirrored-edge MHC. Cosmetic; optional.

2.3.5 NOTE

N1 — Headroom (values > 1 kept) is correct and intentional for an FP pipeline with a separate clip mask; not a bug (it is *more* faithful than DNG clip-to-1). Documented as such in code.

N2 — reconstruct_highlights quality is bounded by white-level accuracy. `peak = max(R,G,B)` is only trustworthy if the brightest channel's white level is right (ties to H1). The method is otherwise a sound, conservative default that correctly preserves genuine single-channel saturated colors and fixes magenta neutrals; it intentionally does **not** do spatial color propagation (the code comment says so). Consider, later, an LCH-blend or guided color-propagation pass for structure recovery in large blown regions.

N3 — WB G2 fallback (`g2 = m[3] != 0 ? m[3] : g, lib.rs:196`) and the `(white-black).max(1.0)` guard (`lib.rs:169`) are sensible defensive defaults; correct.

N4 — data_maximum is correctly NOT used here: it is only populated at `raw2image/dcraw_process`, which this unpack-only flow never calls; relying on it would read a stale/zero value.

2.4 4. Verdict Summary

#	Item	Verdict
2.1	Black level: <code>cblack</code> pattern	Incorrect / Incomplete (pattern ignored)
2.2	White level: scalar maximum vs <code>linear_max</code>	Questionable
2.3	Normalization formula	Correct-with-caveats
2.4	WB before demosaic	Correct
2.5	MHC 5×5 kernels (element-by-element)	Correct
2.6	2-px MHC bilinear border	Correct-with-caveats
2.7	Highlight reconstruction	Correct-with-caveats
2.8a	<code>channel_at</code> mapping / all 4 phases	Correct
2.8b	<code>is_rgb_bayer</code> sensor-type guard	Incorrect (X-Trans accepted)
2.9	Bilinear demosaic & borders	Correct
2.10	FFI / unpack lifecycle	Correct

2.5 5. References

Primary papers / specs (downloaded): - Malvar, He & Cutler, *High-Quality Linear Interpolation for Demosaicing of Bayer-Patterned Color Images*, IEEE ICASSP 2004. Fig. 2 verified element-by-element. → `docs/demosaic-malvar-he-cutler-2004.pdf` (source: https://stanford.edu/class/ee367/reading/Demosaicing_ICASSP04.pdf; Microsoft Research listing: <https://www.microsoft.com/en-us/research/publication/high-quality-linear-interpolation-for-demosaicing-of-bayer-patterned-color-images>)

[quality-linear-interpolation-for-demosaicing-of-bayer-patterned-color-images/](#)) - Adobe, *Digital Negative (DNG) Specification, Version 1.6.0.0*, December 2021. Chapter 5 “Mapping Raw Values to Linear Reference Values”; BlackLevel, BlackLevelRepeatDim, WhiteLevel tag definitions. → docs/demosaic-dng-spec-1.6.0.0.pdf (source: https://paulbourke.net/dataformats/dng/dng_spec_1_6_0_0.pdf)

LibRaw: - *Data Structures and Constants* (libraw_colordata_t: cblack layout, black, maximum, linear_max, data_maximum, cam_mul, cam_xyz; filters==9 ⇒ X-Trans): <https://www.libraw.org/docs/API-datastruct-eng.html> - Forum — *Maximum/saturation pixel value*: <https://www.libraw.org/node/2238> - Forum — *color.maximum and camera white level*: <https://www.libraw.org/node/2473> - Forum — *Pink Clouds with LibRaw 0.20.0* (per-channel white level ↔ magenta highlights; “use linear_max”): <https://www.libraw.org/node/2605> - Forum — *Questions about black-level and white-levels in colordata*: <https://www.libraw.org/node/2565> - Forum — *Determining CFA Pattern from cdesc and filters* (cdesc==“RGBG” for all Bayer orders): <https://www.libraw.org/node/2561> - Forum — *Understanding highlight modes* (dcraw -H ladder): <https://www.libraw.org/node/2694> - Issue #107 — *Black level of FUJIFILM X-Trans detected as 0* (black in cblack pattern): <https://github.com/LibRaw/LibRaw/issues/107> - Forum — *Fujifilm Pattern with rawpy*: <https://www.libraw.org/node/2301>

Demosaic / highlight references: - IPOL, *Malvar-He-Cutler Linear Image Demosaicking* (independent kernel reproduction): http://www.ipol.im/pub/art/2011/g_mhcd/revisions/2011-08-14/g_mhcd.htm - darktable manual — white balance module (WB applied before demosaic): <https://docs.darktable.org/usermanual/development/en/module-reference/processing-modules/white-balance/> - darktable manual — demosaic module: <https://docs.darktable.org/usermanual/development/en/module-reference/processing-modules/demosaic/> - RawPedia — *Exposure* (Highlight Reconstruction: Luminance Recovery, CIELab, Color Propagation): <https://rawpedia.rawtherapee.com/Exposure> - RawTherapee issue #3311 — Color Propagation highlights / purple fringing: <https://github.com/Beep6581/RawTherapee/issues/3311> - dcraw man page (-H 0..9, -G LCH blend): <https://www.dechifro.org/dcraw/dcraw.1.html> - dcraw LCH-blend highlight recovery patch: http://people.zoy.org/~cyril/dcraw_lchblend/highlight_recovery_dcraw_lch_patch.html

Chapter 3

Audit 03 — Tone & Color-Grading Math

Component: latent-image (tone/color), latent-pipeline (lowering) **Files audited:** latent-image/src/tone.rs, latent-image/src/color.rs, latent-pipeline/src/lib.rs (tone_curves, channel_curves, point_curve, apply_global), with cross-reference to latent-edit/src/lib.rs (parameter types/ranges), latent-app/src/gui.rs (slider ranges), and latent-gpu/src/map_pixels.wgsl (GPU tone/saturation path). **Date:** 2026-06-27 **Method:** Each file read line-by-line. Every curve polynomial and the LUT evaluator were **re-implemented in standalone Python** and evaluated on fine grids to *measure* (not assert) monotonicity, endpoint slopes, headroom extrapolation, and LUT-interpolation error; the analytic derivatives were derived by hand and cross-checked against the numeric slopes. Color-science claims were verified against Poynton's *Gamma FAQ* and *Color FAQ* (read element-by-element from the downloaded PDFs), the darktable and Lightroom documentation, and the ICtCp/IPT literature. PDFs cited are downloaded under docs/ with a tone- prefix.

3.1 1. Summary of the Tone / Color Model

latent shapes tone in a **perceptual domain** built from a plain power law. A linear-light value L is encoded with $E(L) = L^{1/\gamma}$, $\gamma = 2.2$ (tone_encode, no upper clamp so headroom $L > 1$ survives), a 1-D curve is applied, then decoded with $D(E) = E^\gamma$ (apply_linear). Curves are stored as a **256-entry LUT** sampled uniformly over $[0, 1]$; eval does linear interpolation inside $[0, 1]$, clamps the low end to 0, and **extrapolates above 1 with the LUT's end slope** so highlight headroom is shaped rather than flattened.

Four parametric tonal shapes, each $\text{clamp}_{[0,1]}$ of a base polynomial with amount a :

$$\begin{aligned}\text{contrast}(t) &= t + a(\text{ss}(t) - t), & \text{highlights}(t) &= t + a t^2(1 - t), \\ \text{shadows}(t) &= t + a t(1 - t)^2, & \text{blacks}(t) &= t + \frac{a}{4}(1 - t)^4,\end{aligned}$$

where $ss(t) = t^2(3 - 2t)$ is the cubic smoothstep. The pipeline composes the active shapes in the fixed order contrast $\rightarrow$ highlights $\rightarrow$ shadows $\rightarrow$ blacks, each applied per channel through the same encode/curve/decode path (`tone_curves`, `apply_global`).

Color tools operate in the linear working space (linear ProPhoto primaries at D65):

- **Saturation:** luma-blend $C' = Y + a(C - Y)$ per channel, clamped ≥ 0 , with $Y = \langle 0.2788, 0.7211, 0.000113 \rangle \cdot C$ (the Y row of the working RGB $\rightarrow$ XYZ matrix).
- **HSL mixer (color_mix):** convert to HSV with $V = \max$ channel, pick the two nearest of 8 evenly-spaced hue bands by linear interpolation, skip achromatic pixels, and apply $h' = h + \Delta h$, $s' = s(1 + \Delta s)$, $v' = v(1 + \Delta v)$.
- **Channel curves:** per-channel = channel-curve $\circ$ master-curve, both piecewise-linear interpolations of control points in the perceptual domain.
- **Channel mixer:** a raw 3×3 matrix (no row normalization at the op level).

Bottom line. The tone/color math is broadly sound and the design choices (perceptual-domain shaping, value=max HSV for headroom, derived luma weights) are defensible and well-cited in the code comments. The headroom *plumbing* is real but the shaped curves do **not** all preserve headroom equally: positive **contrast** and **highlights** flatten to a near-zero top slope, so highlights above 1 are compressed toward white (a soft clip), the opposite of the design's stated intent for those values. The four shapes are monotone for the documented amount range $[-1, 1]$, but **contrast is non-monotone for $a > 1$** — unreachable from the UI slider (clamped to ± 1) yet reachable through the public API / sidecar, where the type is an unclamped f32. Secondary issues: luma-blend saturation in wide D65-ProPhoto primaries drives saturated blues toward black; the HSV hue interpolation and `hsv_to_rgb` boundary work but are not hue-uniform.

3.2 2. Point-by-Point Verification

3.2.1 2.1 Gamma 2.2 as the “perceptual domain” for tone shaping

Code (`tone.rs`:16-26, 69-74): $E(L) = \max(0, L)^{1/2.2}$, $D = E^{2.2}$, curve applied between them.

Authoritative position. Poynton defines lightness as the CIE L^* function

$$L^* = 116 \left(\frac{Y}{Y_n} \right)^{1/3} - 16, \quad \frac{Y}{Y_n} > 0.008856,$$

and states directly: “Video systems approximate the lightness response of vision using $R'G'B'$ signals that are each subject to a 0.45 power function. This is comparable to the 1/3 power function defined by L^* .” (Color FAQ Q4.) On whether to process in a nonlinear domain at all: “if your computation involves human perception, a nonlinear representation may be required ... you ought to use nonlinear coding that exhibits perceptual uniformity, because you wish to minimize the perceptibility of the errors that will be introduced during quantization.” (Gamma FAQ Q19.) And “gamma correction in video effectively codes into a perceptually uniform domain.” (Gamma FAQ Q14.)

So a ~ 0.45 encode (display $\gamma \approx 2.2$) is the *standard* perceptual proxy, explicitly “comparable to” L^* ’s cube root. Shaping contrast/tone in this domain rather than linear light is therefore correct practice, and matches what display-referred photo tools do (Lightroom’s parametric tone curve and point curve act on gamma-encoded values; darktable’s legacy tone curve and color zones operate on non-linear Lab/encoded data). The one nuance Poynton flags (Gamma FAQ Q6) is that a **pure power** differs from the sRGB/Rec.709 OETF by the latter’s *linear toe* near black — sRGB is a $\sim 1/2.4$ power with a linear segment below ≈ 0.0031 , asymptotically matching an overall ≈ 2.2 . The difference vs a pure 2.2 is confined to deep shadows and is small for tone-shaping; it is not a true perceptual space (neither is sRGB — L^* is the reference), but it is the conventional and adequate choice.

Artifacts of the choice. Shaping in $\gamma 2.2$ vs linear: contrast/S-curves in linear light crush shadows and blow highlights non-perceptually (equal linear steps are unequal perceptual steps) — the code comment’s justification is correct. Shaping in $\gamma 2.2$ vs true L^* : the pivot of an S-curve and the placement of “midtones” differ slightly (mid-gray $0.18 \text{ lin} \rightarrow 0.466$ at $\gamma 2.2$ vs 0.5 at L^*), and saturated hues can shift because per-channel power curves are not luminance/hue-preserving — but these are second-order and shared by virtually all per-channel tone tools.

VERDICT: Correct-with-caveats. Gamma 2.2 is a legitimate, standard perceptual proxy for tone shaping (Poynton: comparable to L^* ’s cube root); it is not perceptually *uniform* the way L^*/ICtCp are, and it omits sRGB’s near-black linear toe, but those gaps are minor for the contrast/HLS shapes used here. **Citations:** Poynton, *Color FAQ* Q4 (definition of L^* ; “comparable to the 1/3 power”), Q6; *Gamma FAQ* Q6 (Rec.709 OETF with toe), Q14, Q19.

3.2.2 2.2 Monotonicity of the four shapes over the documented amount range

A tone curve that decreases anywhere inverts tones locally — a real bug. The base polynomials’ derivatives (before the clamp) are:

$$\begin{aligned} \text{contrast}'(t) &= 1 + a(6t - 6t^2 - 1), & \text{highlights}'(t) &= 1 + a(2t - 3t^2), \\ \text{shadows}'(t) &= 1 + a(1 - 4t + 3t^2), & \text{blacks}'(t) &= 1 - a(1 - t)^3. \end{aligned}$$

Minimizing each bracket over $[0, 1]$ gives the monotonicity threshold:

shape	min / max of the bracket on $[0, 1]$	monotone iff
contrast	bracket $\in [-1, 0.5]$ (min -1 at $t = 0, 1$)	$-2 \leq a \leq 1$
highlights	bracket $\in [-1, 1/3]$ (min -1 at $t = 1$)	$-3 \leq a \leq 1$
shadows	bracket $\in [-1/3, 1]$ (min $-1/3$ at $t = 2/3$)	$-1 \leq a \leq 3$

shape	min / max of the bracket on $[0, 1]$	monotone iff
blacks	$(1 - t)^3 \in [0, 1]$	$a \leq 1$ (and $a \geq$ any neg)

Numeric confirmation (Python re-implementation, 2001-point grid, reporting the minimum forward difference of the *clamped* shape; a negative value means a local inversion):

contrast	a=-1.00: min_slope=+2.50e-04	a=+1.00: min_slope=+7.50e-07
highlights	a=-1.00: min_slope=+3.33e-04	a=+1.00: min_slope=+5.00e-07
shadows	a=-1.00: min_slope=+5.00e-07	a=+1.00: min_slope=+3.33e-04
blacks	a=-1.00: min_slope=0 (clamp)	a=+1.00: min_slope=+3.75e-07

All four are **monotone (non-decreasing) for every $a \in [-1, 1]$** — the documented range (SelectiveTone doc: “*each value is roughly $[-1, 1]$* ”). At the endpoints $a = \pm 1$ the slope touches ≈ 0 but never goes negative, so there is no inversion within range.

Past the documented range — contrast inverts for $a > 1$:

```
contrast a=+1.00: raw_min_slope=+0.000000 mono=True
contrast a=+1.50: raw_min_slope=-0.000125 mono=False <-- inversion
contrast a=+2.00: raw_min_slope=-0.000250 mono=False
contrast a=+2.50: raw_min_slope=-0.000375 mono=False
```

This matches the analytic threshold exactly: $\text{contrast}'(t) = 1 + a(6t - 6t^2 - 1)$, whose minimum over $[0, 1]$ is $1 - a$ (at $t = 0, 1$), so the curve is monotone iff $a \leq 1$. For $a > 1$ the curve **decreases near both endpoints** — i.e. raising contrast past 1 makes near-black get *brighter* and near-white get *darker*, a tone inversion. The GUI slider is clamped to $-1.0 \dots 1.0$ (latent-app/src/gui.rs:569), so this is not reachable interactively, **but** SelectiveTone.contrast is an unclamped f32 deserialized from sidecars / set via the public API, and tone::contrast’s doc comment promises it “returns a monotonic curve.” The clamp lives in the UI, not in the math.

Endpoints / C1. $ss(0) = 0, ss(1) = 1$ and the highlights/shadows/blacks terms vanish at the relevant endpoint, so $f(0) = 0$ (except blacks, which intentionally lifts/lowers the floor) and $f(1) = 1$ — endpoints behave as documented. Each base polynomial is a smooth (C^∞) polynomial, so individually C^1 ; but the **clamp** introduces a slope discontinuity wherever the curve hits 0 or 1 (e.g. blacks(-1) shows min_slope=0 exactly because the clamp flattens the toe), and **composing** several shapes in series, plus the LUT’s piecewise-linear interpolation, makes the realized transfer function only C^0 overall. For tone shaping this is acceptable (the kink is gentle and far from typical operating points), but it is not the C^1 that “smoothstep” might suggest end-to-end.

VERDICT: Correct (within documented $[-1, 1]$) / Questionable at the boundary (contrast, $a > 1$). No inversion can occur for any in-range amount; the only inversion is contrast for $a > 1$, which the UI prevents but the type system and tone::contrast’s contract do not. **Citations:** standard monotone-tone-curve requirement (a tone curve must be non-decreasing); analytic derivatives above, confirmed numerically.

3.2.3 2.3 Headroom: end-slope extrapolation of the LUT above 1.0

eval extrapolates linearly past $t = 1$ using the top LUT slope $s = (\text{lut}[255] - \text{lut}[254]) \cdot 255$ (tone.rs:53-59). apply_linear reaches this whenever $L > 1$, since $E(L) = L^{1/2.2} > 1$. The realized highlight behavior depends entirely on each shape’s **slope at $t = 1$** :

$$\text{contrast}'(1) = 1 - a, \quad \text{highlights}'(1) = 1 - a, \quad \text{shadows}'(1) = 1, \quad \text{blacks}'(1) = 1.$$

Measured top LUT slope and apply_linear on a highlight ramp (linear inputs 1.0, 1.5, 2.5, 8.0):

shape	a	top_slope	apply_linear(1.0, 1.5, 2.5, 8.0)				mono	neg
contrast	+1.00	+0.0117	1.0000	1.0052	1.0134	1.0411	yes	no
highlights	+1.00	+0.0078	1.0000	1.0035	1.0089	1.0273	yes	no
contrast	-1.00	+1.9883	1.0000	2.1043	4.7335	22.6294	yes	no
highlights	-1.00	+1.9922	1.0000	2.1069	4.7438	22.7036	yes	no
shadows	+1.00	+0.9961	1.0000	1.4978	2.4927	7.9580	yes	no
shadows	-1.00	+1.0039	1.0000	1.5022	2.5073	8.0421	yes	no
blacks	+/-1.00	+1.0000	1.0000	1.5000	2.5000	8.0000	yes	no

Findings:

1. **No non-monotonicity, no negatives, no runaway** for any shape at $a \in [-1, 1]$ on the headroom range — the extrapolation is well-behaved in sign and order. Good.
2. **Positive contrast / highlights crush the headroom they are supposed to shape.** Because $f'(1) = 1 - a \rightarrow 0$ as $a \rightarrow 1$, the top LUT slope is ≈ 0.01 , so a linear value of 8.0 maps to only ≈ 1.04 — a *soft clip* of all highlight headroom to just above white. This is the **opposite** of the module’s stated design: tone_encode’s comment says headroom “stays above 1.0 ... so the curve can shape them instead of crushing to white,” and eval’s comment says headroom is “shaped, not flattened.” For positive contrast/highlights it is in fact flattened. (For *negative* contrast/highlights, $f'(1) = 1 - a = 2$, so headroom is *expanded* $\sim 2\times$ — also a strong, possibly surprising effect: $8.0 \rightarrow 22.6$.)
3. **Slope-1 shapes pass headroom through faithfully.** shadows and blacks have $f'(1) = 1$, so $L = 8 \rightarrow \approx 8$ — headroom preserved as intended.

The root cause is that the extrapolation uses the **local slope at the very top of $[0, 1]$** , which for highlight-targeting shapes is exactly where the curve has been bent toward flat. A more headroom-faithful design would extrapolate with the curve’s *average* high-end slope, or apply the highlight/contrast bend only up to 1 and pass $L > 1$ through with unit slope, or shape headroom with an explicit highlight-rolloff rather than a linear extension of a near-zero slope.

VERDICT: Correct (numerically safe — monotone, non-negative, bounded) / Questionable (semantically). The extrapolation never produces an inverted, negative, or runaway value, so it is a *sound* mechanism; but for positive contrast and positive highlights it compresses highlight headroom to a near-flat plateau, contradicting the documented “shape, don’t crush” intent. The same logic is replicated in

the GPU shader (`map_pixels.wgsl:38-40`), so both backends behave identically. **Citations:** module doc comments (`tone.rs:18-22,53-59`); numeric evidence above; analytic endpoint slopes.

3.2.4 2.4 256-sample LUT with linear interpolation — banding / quantization

Measured maximum interpolation error of the 256-entry LUT vs the exact shape (worst amount $a = 1$, sampled at 20 001 points), reported in 16-bit and 8-bit levels:

shape	max LUT - exact	16-bit levels	8-bit levels
contrast	1.149e-05	0.753	0.0029
highlights	7.667e-06	0.502	0.0020
shadows	7.667e-06	0.502	0.0020
blacks	5.744e-06	0.376	0.0015
identity	2.2e-16	~0	~0

The worst-case error (contrast, ≈ 0.75 of a 16-bit level) is **below 1 LSB at 16-bit** and $\sim 1/340$ of an 8-bit level. With $N = 256$ samples a piecewise-linear LUT has error $\leq \frac{1}{8} \max |f''| h^2$ with $h = 1/255$; the smooth shapes have small $|f''|$, so the bound is tiny. Poynton notes 8-bit *linear* coding bands near black, but here the LUT is in the **already-perceptual** domain and feeds 16-bit output, so the perceptual bit-budget is well spent (Gamma FAQ Q13: nonlinear coding needs ~ 9 bits for smooth black-to-white; 16-bit output has ample margin). One caveat: the LUT samples are stored as f32 and interpolated in f32, so there is no added quantization beyond the 0.75-LSB geometric error — banding risk is negligible.

VERDICT: Correct. 256 entries with linear interpolation in the perceptual domain is sufficient for 16-bit output; worst-case error is sub-LSB at 16-bit. **Citations:** Poynton, *Gamma FAQ Q13* (bit-depth for smooth shading); piecewise-linear interpolation error bound; numeric evidence above.

3.2.5 2.5 color_mix — 8-band HSV hue grading

Code (`color.rs:234-253`). For a chromatic pixel: $\text{pos} = 8h$, $i = \lfloor \text{pos} \rfloor \bmod 8$, $j = (i + 1) \bmod 8$, $f = \text{frac}$, $\text{adj} = \text{bands}[i](1 - f) + \text{bands}[j]f$; then $h' = h + \Delta h$, $s' = (s(1 + \Delta s))_{[0,1]}$, $v' = (v(1 + \Delta v))_{\geq 0}$.

Soundness. Two-neighbor linear interpolation over evenly-spaced band centers is exactly how Lightroom’s HSL/Color mixer behaves (“the target tool ... will adjust sliders in both colors at the same time if you click on part of your image that contains both yellows and greens”; 8 bands red/orange/yellow/green/aqua/blue/purple/magenta). **Wraparound** band 7 $\leftrightarrow$ 0 is handled by the `%8` on j (`color.rs:243`): at $h = 0.9375$ the weights split 0.5/0.5 between band 7 and band 0 — correct and continuous across the seam (confirmed numerically). Achromatic pixels ($s \leq 10^{-6}$) are skipped so neutrals don’t fall into the red band at $h = 0$ — a correct and deliberate choice (tested). The all-zero-adjacent-bands early-out keeps untouched hues exact.

Two real caveats:

1. **Band centers sit at $h = i/8$ (the lower edge), not $(i + 0.5)/8$.** With $\text{pos} = 8h$ and $i = \lfloor \text{pos} \rfloor$, band i has full weight ($f = 0$) exactly at $h = i/8$. So band 0 (“red”) is centered on pure red ($h = 0$), band 4 on cyan/aqua ($h = 0.5$), etc. The doc says “a color at a band center is driven only by that band,” which is true at $h = i/8$ — consistent, but worth stating explicitly since “band center” could be misread as the middle of a sector.
2. **HSV (value = max channel) is not a hue-preserving / perceptual space.** Poynton is blunt: of HSI/HSV/HLS, “no reference is made to the nonlinearity in the primary components ... these systems make poor measures of perceptual quantities” (Gamma FAQ Q20), and the saturation/value of HSV are not the CIE attributes. Scaling HSV “value” (the max channel) as a luminance proxy will shift perceived lightness differently for different hues, and HSV hue is not perceptually uniform (equal Δh are unequal perceptual hue steps, worst in blues). The professional comparison point is **darktable color zones**, which works in **CIE LCh** (“selectively adjusts the lightness, chroma and hue ... working in CIE LCh”), a hue-preserving space; Lightroom’s calibration/HSL is a proprietary perceptual model. Using HSV is the *common, cheap* choice and is internally consistent, but it is a “HSL mixer” in name more than in colorimetric behavior. Note also the tool is labeled **HSL** in the UI/type (Hsl) yet implemented in **HSV** — the “lum” control scales HSV *value* (max channel), not HSL *lightness* $(\text{max} + \text{min})/2$; these diverge for saturated colors.

VERDICT: Correct-with-caveats. The band interpolation, wraparound, and neutral-skip are sound and match Lightroom practice; the space is HSV-not-HSL and not hue-uniform (vs darktable’s LCh), so hue/lightness behavior is approximate. **Citations:** Poynton, *Gamma FAQ Q20* (HSI/HSV lack perceptual basis); darktable *color zones* (CIE LCh); Lightroom Color Mixer docs (8 bands, two-band interpolation).

3.2.6 2.6 rgb_to_hsv / hsv_to_rgb correctness and round-trip (with headroom)

Code (color.rs:192-225). Standard hexagonal HSV: $C = \text{max} - \text{min}$, $S = C / \text{max}$, the six-sector hue formula, and the inverse with $X = C(1 - |h_6 \bmod 2 - 1|)$, $m = V - C$.

Numeric round-trip (re-implemented), including $V > 1$:

```
[0.8,0.3,0.1] -> hsv(0.0476,0.8750,0.8000) -> [0.8000,0.3000,0.1000] err 5.6e-17
[2.0,0.3,0.0] -> hsv(0.0250,1.0000,2.0000) -> [2.0000,0.3000,0.0000] err 5.6e-17
[1.6,0.2,1.6] -> hsv(0.8333,0.8750,1.6000) -> [1.6000,0.2000,1.6000] err 5.6e-17
[3.0,3.0,1.0] -> hsv(0.1667,0.6667,3.0000) -> [3.0000,3.0000,1.0000] err 0
[0.0,0.0,0.0] -> hsv(0,0,0) -> [0,0,0] err 0
```

Round-trip is exact to float epsilon, and $V > 1$ **headroom reconstructs faithfully** because $V = \text{max}$ carries the unbounded range and the inverse scales by V . This

is the correct design choice for a headroom pipeline (HSL lightness $(\max + \min)/2$ would also carry headroom but distorts saturation differently; the code comment’s justification is right). The forward formula uses `rem_euclid` so negative-ish hues wrap correctly; the $c \leq 10^{-9}$ guard avoids a divide-by-zero on neutrals.

One latent boundary subtlety (benign). `hsv_to_rgb` casts `h6` as `u32` to pick the sector (`color.rs:216`). At exactly $h = 1.0$, $h_6 = 6.0$, 6 as `u32` = 6, which falls to the `_` arm ($c, 0, x$); there $h_6 \bmod 2 = 0 \Rightarrow X = 0$, giving $(c, 0, 0)$ — identical to sector-0’s $(c, x, 0) = (c, 0, 0)$, so the result is correct **by coincidence** (verified: $h = 1.0 \rightarrow [1, 0, 0]$). The `rem_euclid(1.0)` on the way in normally keeps $h \in [0, 1)$, so $h_6 \in [0, 6)$ and the `_` arm is only legitimately reached for $h_6 \in [5, 6)$; the $h_6 = 6.0$ case is reachable only if a caller passes an un-normalized hue and the float lands exactly on 1.0. No bug, but the cast relies on this coincidence rather than an explicit clamp to sector 0..5.

VERDICT: Correct. Standard hexagonal HSV, exact round-trip, headroom preserved via $V = \max$. The $h = 1.0$ sector cast is coincidentally correct and never wrong in practice. **Citations:** standard HSV/HSV-inverse formulas (hexagonal model); code round-trip test corroborated numerically.

3.2.7 2.7 Saturation op — luma-blend with working-space weights

Code (pipeline `lib.rs:386-388`, backend `:810-817`; GPU `:75-76`): $C'_k = \max(0, Y + a(C_k - Y))$, $Y = \langle 0.2788, 0.7211, 0.000113 \rangle \cdot C$.

Is luma-blend saturation standard? Yes — interpolating between luminance and color ($a = 0$ gray, $a = 1$ identity, $a > 1$ more saturated) is the textbook “saturation as a lerp toward gray” and is what many engines do (it is the same construction as a saturation matrix). Using **true relative-luminance weights** for the *working* primaries (the Y row of working→XYZ, cross-checked in tests) is more correct than the common shortcut of reusing Rec.709 weights in a non-709 space. So far, correct.

The blue problem. Poynton: “all saturated blue colors are quite dark ... the blue coefficient will be the smallest of the three” (Color FAQ Q9), and in these *wide ProPhoto* primaries the blue luminance weight is ≈ 0.000113 — essentially zero (the code comment acknowledges this). Consequences, confirmed numerically:

```
pure blue [0,0,1], amount=0 (desaturate) -> luma 0.00011 -> [0.0001,0.0001,0.0001] (~BLACK)
pure blue [0,0,1], amount=2 (boost)      -> [0.0000,0.0000,1.99989]
[0.1,0.1,1.0],    amount=1.5            -> [0.0999,0.0999,1.4500]
```

So **desaturating a pure blue sends it to near-black**, not to a mid-gray, because the gray it blends toward is $Y \approx 0$. Under sRGB/Rec.709 primaries the same blue would desaturate to ≈ 0.0722 (a dark gray), and in a chroma-preserving space (Lab/LCh: drop chroma at constant L , or ICtCp: scale C_t, C_p at constant I) it would desaturate to a *mid* gray at the blue’s own lightness. The code comment frames this as “by design, not a bug ... colorimetrically correct,” and it is colorimetrically correct as a luminance — but as a **saturation control** it is perceptually wrong: lowering saturation should not also crater lightness. Note also that for $a > 1$ the op can push channels below 0

(hence the $\max(0, \cdot)$ clamp), which itself shifts hue/luminance — boosting saturation of an already-saturated color clips and desaturates the *other* channels asymmetrically.

This is the same class of issue darktable/Capture One avoid by doing saturation/vibrance in a chroma-preserving space rather than a luma lerp; it is the most defensible “Medium” finding here because the visual result (blues going muddy/black when desaturated, and the global saturation slider darkening blue-heavy images) is real and user-visible.

VERDICT: Correct-with-caveats. Luma-blend saturation with derived working-space weights is a standard, internally-consistent construction; but the near-zero blue weight of wide D65-ProPhoto primaries makes desaturation collapse blues toward black and saturation changes shift blue lightness — a chroma-preserving space (LCh/ICtCp) would avoid the coupling. **Citations:** Poynton, *Color FAQ* Q6 (saturation = colorfulness in proportion to brightness), Q9-Q10 (blue carries least luminance; blue contouring); LUMA_WEIGHTS comment (color.rs:177-181).

3.2.8 2.8 channel_curves — composition order

Code (pipeline lib.rs:592-599): `ToneCurve::from_fn(|t| channel.eval(master.eval(t)))`.

This evaluates **master first, then the per-channel curve** on the master’s output: $\text{eff} = \text{channel} \circ \text{master}$, i.e. $\text{eff}(t) = \text{channel}(\text{master}(t))$. The doc states the per-channel curve is “*composed after the master*” and “*master then per-channel*” — function composition $g \circ f$ means “apply f (master) then g (channel),” so the code matches the documented intent exactly. This mirrors Photoshop/Lightroom Curves, where the RGB (master) curve applies and the per-channel curves act on the result. Both point_curves are clamped-flat piecewise-linear interpolations of sorted control points (lib.rs:604-623), C^0 , identity when empty — standard and correct. (Piecewise-linear point curves are only C^0 , so they produce slope discontinuities at control points — acceptable for a curves tool and matching most implementations’ linear/as-shot interpolation; a Catmull-Rom/monotone-cubic spline would be smoother but is a different design choice.)

VERDICT: Correct. Composition order is master-then-channel, matching both the doc and standard Curves behavior. **Citations:** function-composition semantics; Adobe Curves master+per-channel model.

3.2.9 2.9 Channel mixer — raw 3×3, not row-normalized at the op

Code (pipeline lib.rs:392-393, op Matrix): applies `cm.matrix` directly; `ChannelMixer::default` is identity (latent-edit/src/lib.rs:383-388). A `row_normalized()` exists in `Mat3` but is **not** applied here (it is used for the *camera*→*working* and *working*→*sRGB* matrices, where neutral-preservation is required).

Neutral-shift behavior. A channel mixer whose rows do **not** each sum to 1 will shift a neutral input: $[\nu, \nu, \nu] \mapsto [\nu \Sigma_R, \nu \Sigma_G, \nu \Sigma_B]$ where Σ_c is row c ’s sum. This is the **stan-**

dard, intended behavior of a creative channel mixer — Photoshop/Lightroom expose a per-row “constant”/total and let the user create tints or a custom monochrome by deliberately un-balancing rows; row sums $\neq 1$ are how you warm/cool or build a B&W mix. So *not* normalizing is correct for a creative tool (unlike the color-management matrices, which must stay neutral). The only caveat worth a Note: the default-on UX should make clear that a row summing to $\neq 1$ tints neutrals, and a “preserve luminosity”/normalize toggle (as Photoshop offers) would be a friendly addition. The op itself is a plain, correct $M \cdot v$ (verified by the swap/identity tests).

VERDICT: Correct. A raw, un-normalized 3×3 is the right primitive for a creative channel mixer; the neutral shift for non-unit row sums is the expected, desired behavior, distinct from the color-management matrices that *are* row-normalized. **Citations:** Photoshop / GIMP Channel Mixer semantics (per-channel totals, optional “monochrome”/preserve-luminosity); `Mat3::row_normalized` usage in `color.rs` (only for CM matrices).

3.3 3. Findings by Severity

#	Severity	Location	Issue
F1	High	<code>tone.rs:94-95</code> (contrast); <code>latent-edit/src/lib.rs:405</code> (<code>SelectiveTone.contrast</code> <code>f32</code>)	contrast is non-monotone for $a > 1$ (tone inversion near black and white). The UI clamps to ± 1 (<code>gui.rs:569</code>) but the type/API/sidecar are unclamped, and <code>tone::contrast</code> ’s doc promises a monotonic curve.

#	Severity	Location	Issue
F2	High	tone.rs:53-59 (eval extrapolation) + tone.rs:100-101 (highlights), 94-95 (contrast); GPU map_pixels.wgsl:38- 40	Positive con- trast/highlights have $f'(1) = 1 - a \rightarrow 0$, so the LUT end-slope extrapolation compresses highlight headroom to a near-flat plateau ($L = 8 \rightarrow \approx 1.03$ - 1.04). This contradicts the documented “shape, don’t crush” headroom intent for exactly the highlight-targeting controls. Numerically safe (monotone, non-negative) but semantically a soft clip.
F3	Medium	color.rs:181-186, pipeline lib.rs:386-388, backend :810-817, GPU :75-76	Luma-blend saturation in wide D65-ProPhoto: blue luma weight ≈ 0.0001 , so desaturating blue → near-black and saturation changes shift blue lightness. Colorimetrically a correct luminance, but perceptually wrong for a saturation control.

#	Severity	Location	Issue
F4	Medium	color.rs:211-253; type Hsl (latent- edit/src/lib.rs:368)	“HSL” tool is implemented in HSV (value = max channel), and the space is not hue-uniform. The “lum” slider scales HSV <i>value</i> , not HSL <i>lightness</i> ; hue steps are non-uniform (worst in blue). darktable’s equivalent uses CIE LCh.
F5	Low	tone.rs:95,101,107, (clamp) + composition in tone_curves/channel_pieces	Realized transfer is only C^0 , not C^1 : the clamp(0,1), the piecewise-linear LUT, and piecewise-linear point_curves introduce slope kinks (e.g. blacks(-1) flattens at the floor). Gentle and far from typical operating points, but “smoothstep” implies more smoothness than the end-to-end curve has.

#	Severity	Location	Issue
F6	Note	tone.rs:18-22 (pure y2.2)	The perceptual domain is a pure power , omitting sRGB/Rec.709's near-black linear toe ; differs from sRGB only in deep shadows. Adequate, but not identical to the standard OETF and not perceptually uniform like L^* .
F7	Note	color.rs:216 (h6 as u32)	hsv_to_rgb sector cast is coincidentally correct at $h = 1.0$ (falls to the <code>_arm</code> , which equals sector 0 there). Relies on <code>rem_euclid</code> + the $X = 0$ coincidence rather than an explicit 0..5 clamp.
F8	Note	color.rs:241-245	<code>color_mix</code> band centers sit at $h = i/8$ (the lower sector edge), so “band center” means the band's pure hue, not the middle of a 1/8-turn sector. Consistent with the doc but easy to misread.

3.3.1 Recommendations

- **F1 (High):** Clamp/guard contrast (and ideally all four shapes) to their monotone range inside `tone::contrast` — e.g. `let amount = amount.clamp(-2.0, 1.0);` — or document the contract as “monotone for $a \in [-1, 1]$ ” and clamp on deserialization of `SelectiveTone`. The math contract should not depend on the UI slider's range. A non-clamping alternative for $a > 1$ is to switch to a strictly-monotone S-curve family (e.g. a logistic or a power-pivot contrast) whose slope stays positive

for all a .

- **F2 (High):** Decide what positive contrast/highlights should do above white. Options: (a) apply the contrast/highlights bend only on $[0, 1]$ and pass $L > 1$ through with **unit slope** (preserve headroom); (b) extrapolate with the curve's **high-end average slope** rather than the very-top local slope; (c) replace the linear extension with an explicit highlight **rolloff** so headroom compresses smoothly and intentionally. At minimum, correct the `tone.rs` comments, which currently claim headroom is “shaped, not flattened” when positive contrast/highlights flatten it.
- **F3 (Medium):** Offer (or switch to) a **chroma-preserving** saturation — scale chroma at constant lightness in Lab/LCh, or scale C_t, C_p at constant I in ICtCp — so desaturation goes to mid-gray and saturation does not shift blue lightness. If the luma-lerp is kept for speed, document the blue-darkening explicitly in user-facing terms (it is currently only an internal comment).
- **F4 (Medium):** Either rename the tool/type to “HSV mixer,” or reimplement the lightness axis in true HSL lightness (or LCh) so “lum” matches its label; consider an LCh hue path for hue-uniform shifts. At minimum, document the HSV-not-HSL choice in the user-facing tool.
- **F5/F7/F8 (Low/Note):** Optional. For F7, clamp the sector index to $0..=5$ (`(h6 as u32).min(5)`) to make correctness explicit rather than coincidental. For F8, reword “band center” to “band hue ($h = i/8$).”

3.4 4. References

1. **Charles Poynton, *Frequently Asked Questions about Gamma*** — pure-power vs Rec.709 OETF and the near-black linear toe (Q6); bit-depth for smooth shading / banding (Q13); “gamma correction ... codes into a perceptually uniform domain” (Q14); process in linear vs nonlinear domain (Q19); HSI/HSV/HLS lack a perceptual basis (Q20). Downloaded: `docs/tone-poynton-gamma-faq.pdf`. <https://www.poynton.ca/faq/gammafaq/GammaFAQ.pdf>
2. **Charles Poynton, *Frequently Asked Questions about Color*** — definition of lightness $L^* = 116(Y/Y_n)^{1/3} - 16$ and “a 0.45 power function ... comparable to the 1/3 power function defined by L^* ” (Q4); saturation as colourfulness in proportion to brightness (Q6); luma/luminance weights, blue carries least luminance and blue contouring (Q9–Q11). Downloaded: `docs/tone-poynton-color-faq.pdf`. <https://www.poynton.ca/pdf/ColourFAQ.pdf>
3. **IEC 61966-2-1 (sRGB)** — the sRGB OETF: $\sim 1/2.4$ power with a linear segment below ≈ 0.0031 , overall ≈ 2.2 (the “near-black linear toe” of F6). <https://www.color.org/srgb.pdf>
4. **darktable — color zones module** — selective lightness/chroma/hue grading in CIE LCh (the hue-preserving reference for F4). <https://docs.darktable.org/usermanual/development/en/module-reference/processing-modules/color-zones/>
5. **darktable — filmic rgb** — scene-referred linear tone mapping context (S-curve from virtual nodes; logarithmic encode); contrast for display-referred power-domain shaping. <https://docs.darktable.org/usermanual/development/en/module-reference/processing-modules/filmic-rgb/>
6. **Adobe Lightroom — HSL / Color Mixer panel** — eight hue bands (red, orange,

yellow, green, aqua, blue, purple, magenta) with two-band interpolation between sliders (corroborates §2.5). <https://www.capturelandscapes.com/hsl-color-panel-in-lightroom/>

7. **GIMP — Channel Mixer filter** (same creative channel-mixer model as Photoshop): per-output-channel linear mix of inputs with a monochrome/“preserve luminosity” option; un-normalized rows deliberately tint/brighten (corroborates §2.9). <https://docs.gimp.org/2.10/en/gimp-filter-channel-mixer.html>
 8. **ITU-R BT.2100 / ICtCp; Ebner & Fairchild, IPT** — hue-linear, more perceptually uniform color-difference spaces (the “proper chroma-preserving space” reference for F3/F4). <https://en.wikipedia.org/wiki/ICtCp>
 9. **G. M. Johnson et al. / I. Lissner & P. Urban, “A Uniform and Hue-Linear Color Space”** — comparison of CIELAB/CAM16/ICtCp/Jzazbz hue linearity (blue-hue shifts), motivating hue-preserving grading. <https://library.imaging.org/cic/articles/25/1/art00043>
-

3.4.1 Appendix A — Reproducing the numeric checks

The verification scripts are standalone Python (no dependencies), re-implementing `tone_encode/decode`, the 256-entry LUT build/eval (including the > 1 end-slope extrapolation) and `apply_linear`, and the HSV/saturation/color_mix math exactly as in the Rust source. They print: (A) clamped min-slope of each shape on $[0, 1]$ for $a \in \{-1, -0.5, 0.5, 1\}$ (all ≥ 0); (B) contrast raw min-slope vs a (negative for $a > 1$, threshold $a = 1$ matching $\text{contrast}' = 1 - a$); (C) `apply_linear` on the headroom ramp $\{1, 1.5, 2.5, 8\}$ and the top LUT slopes ($1 - a$ for contrast/highlights, 1 for shadows/blacks); (D) the analytic endpoint slopes; (E) the 256-LUT max interpolation error in 16-/8-bit levels (sub-LSB at 16-bit); plus the HSV round-trip with $V > 1$, the saturation blue-collapse, and the band-center / wraparound positions. All numeric outputs quoted above were produced by these scripts.

Chapter 4

Audit 04 — Spatial & Frequency-Domain Filters

Component: latent-pipeline (filter models / lowering) and latent-cpu (the reference implementations) **Files audited:** latent-pipeline/src/lib.rs (lines ~46-567), latent-cpu/src/lib.rs (lines ~65-262), with cross-reference to latent-image/src/color.rs (luminance, LUMA_WEIGHTS) and latent-image/src/tone.rs (GAMMA). **Scope:** The four spatial primitives that run in the GLOBAL stage, in pipeline order **denoise** → **dehaze** → **clarity** → **sharpen** (all before geometry resample): box blur, the two CombineKind recombiners (Unsharp / LocalContrast), the split-channel bilateral denoise, and the dark-channel-prior dehaze. Each is checked for correctness against the primary literature. **Date:** 2026-06-27 **Method:** Code read line-by-line, then every numeric/algorithmic claim verified against primary sources. PDFs downloaded under docs/ with a spatial- prefix: - docs/spatial-bilateral-tomasi-1998.pdf — C. Tomasi, R. Manduchi, *Bilateral Filtering for Gray and Color Images*, ICCV 1998 (8 pp., Stanford author host). - docs/spatial-dehaze-he-2011-tpami.pdf — K. He, J. Sun, X. Tang, *Single Image Haze Removal Using Dark Channel Prior*, **IEEE TPAMI 33(12), 2011** — the extended journal version of the CVPR 2009 paper, by the same authors (13 pp., CUHK mmlab host). All equation numbers below cite this version; the model, prior, $\omega = 0.95$, $t_0 = 0.1$, 15×15 patch, and the soft-matting refinement are identical to CVPR 2009. - docs/spatial-dehaze-ipol-2024.pdf — peer-reviewed IPOL reproduction of the method (companion reference).

Numeric claims were checked with throwaway Python (math only); the snippets and their output are reproduced inline as evidence.

4.1 1. Summary of Each Filter’s Model

4.1.1 1.1 Box blur (latent-cpu blur / blur_axis)

A separable box mean: a horizontal 1-D pass of width $2r + 1$ followed by a vertical one, $r = \text{round}(\text{radius})$, border **clamp-to-edge** (replicate). For a 2-D box this is exact (the

box kernel is rank-1 separable), so the separation is an $O(r)$ optimization with no loss. Radius 0 returns a clone.

$$B_r[f](x) = \frac{1}{2r+1} \sum_{d=-r}^r f(\text{clamp}(x+d)), \quad 2\text{-D} = B_r^{\text{vert}} \circ B_r^{\text{horiz}}.$$

4.1.2 1.2 Unsharp recombine (CombineKind::Unsharp{gain}, lowered by sharpen)

sharpen blurs once to a base $L = B_r[I]$, sets $\text{gain} = 1 + \text{amount}$, and recombines

$$O = L + \text{gain} \cdot (I - L) = L + (1 + a)(I - L).$$

4.1.3 1.3 Local-contrast recombine (CombineKind::LocalContrast{amount}, lowered by clarity)

clarity builds the base from **three** successive box blurs $L = B_r^3[I]$ (central-limit Gaussian approximation), then

$$O = I + a \cdot m(Y(L)) \cdot (I - L), \quad m(\ell) = 1 - (2b - 1)^2, \quad b = \text{clamp}(\ell)^{1/\gamma}, \quad \gamma = 2.2,$$

where Y is relative luminance and m is the midtone window evaluated in the perceptual (gamma-2.2) domain on the **base** luminance.

4.1.4 1.4 Bilateral denoise (bilateral_pixel, lowered by noise_reduction)

Each pixel splits into luminance $Y = Y(\text{rgb})$ and chroma $C = \text{rgb} - Y$ (a per-channel offset). **Two independent** bilateral averages run over the same $\pm r$ window ($r = \text{round}(\text{radius})$), spatial $\sigma_s = r/2$, with **separate** range scales — luma stops on the luma difference, chroma stops on the chroma (vector) difference:

$$w_s(\Delta) = e^{-\frac{\|\Delta\|^2}{2\sigma_s^2}}, \quad Y_{\text{out}} = \frac{\sum w_s e^{-\frac{(Y-Y_n)^2}{2\sigma_l^2}} Y_n}{\sum w_s e^{-\frac{(Y-Y_n)^2}{2\sigma_l^2}}}, \quad C_{\text{out}} = \frac{\sum w_s e^{-\frac{\|C-C_n\|^2}{2\sigma_c^2}} C_n}{\sum w_s e^{-\frac{\|C-C_n\|^2}{2\sigma_c^2}}},$$

recombined $O = Y_{\text{out}} + C_{\text{out}}$. A scale of 0 leaves that component untouched.

4.1.5 1.5 Dark-channel dehaze (dehaze_dark_channel, dehaze_recover, lowered by dehaze)

Atmospheric scattering model $I = Jt + A(1 - t)$ with airlight **fixed at** $A = 1$ (neutral white, *not* estimated). Patch dark channel over a $(2 \cdot 4 + 1)^2 = 9 \times 9$ window (DEHAZE_PATCH=4):

$$dc(x) = \min_{y \in \Omega(x)} \min_{c \in \{r, g, b\}} I^c(y), \quad t = \text{clamp}(1 - \text{strength} \cdot dc, t_0, 1), \quad t_0 = 0.1,$$

$$J^c = \frac{I^c - 1}{t} + 1 \quad (\text{for } I^c \leq 1), \quad \text{headroom } (I^c - 1)_+ \text{ passed through.}$$

strength plays the role of the prior's ω . The raw patch transmission is used directly (no soft-matting / guided-filter refinement).

4.2 2. Point-by-Point Verification

4.2.1 2.1 Unsharp mask — formula and algebra

Code (latent-pipeline/src/lib.rs:438-440, recombine at latent-cpu/src/lib.rs:79-86):

```
let base = backend.blur(&img, s.radius);
let gain = 1.0 + s.amount;
backend.combine(&mut img, &base, &CombineKind::Unsharp { gain });
// combine: *px = o[c] + gain * (px[c] - o[c])
```

Standard. The canonical digital unsharp mask is

$$\text{sharpened} = \text{original} + \text{amount} \cdot (\text{original} - \text{blurred}).$$

Wikipedia states it verbatim: “*sharpened = original + (original – blurred) × amount*” and defines amount as controlling “*the magnitude of each overshoot (how much darker and how much lighter the edge borders become)*.” Imatest gives the equivalent USM form $L_{\text{USM}} = L - k_{\text{USM}} \cdot \text{Blur}$ with a Gaussian blur kernel.

Algebra (verified numerically): substituting $\text{gain} = 1 + a$ into the code form,

$$L + (1 + a)(I - L) = L + (I - L) + a(I - L) = I + a(I - L),$$

which is **exactly** the standard with $\text{amount} = a$. Random-input check (other + (1+amount)(img-other) vs img + amount(img-other)) agreed to $< 10^{-9}$ over many samples.

VERDICT: Correct. The $\text{gain} = 1 + \text{amount}$ parameterization is the standard unsharp mask exactly. The step-edge test (sharpening_overshoots_a_step_edge, lib.rs:1119) confirms the expected dark-side undershoot and bright-side overshoot. **Citations:** Wikipedia, *Unsharp masking* (formula, overshoot/halo semantics); Imatest, *Sharpening* (USM equation, “small overshoots enhance sharpness; large overshoots cause halos”).

Caveat — domain (see §3, Note N1). The blur and recombine run in **linear light**. Blurring/convolution is *physically* correct in linear light (NVIDIA GPU Gems 3, Ch. 24, *The Importance of Being Linear*: light transport is linear, so filtering/blending/mip generation must be linear to be correct). But unsharp **overshoot is asymmetric in**

linear light: because the eye is $\sim$ gamma-2.2, equal linear $\pm\delta$ overshoots around a base look like a much larger bright halo than dark halo, so linear-light sharpening tends to produce **brighter, more visible halos on the dark side of edges** than the same operation in a perceptual/luma domain — the opposite of what most camera/raw sharpeners do (they sharpen the gamma-encoded luma/L channel). This is a *design tradeoff consistent with latent’s linear-light philosophy*, not a bug, but it should be understood. Sharpening all three RGB channels independently (rather than luma only) can also shift hue/saturation at edges (color fringing), where the convention in raw developers is luminance-only sharpening.

4.2.2 2.2 Box blur — separability, border, radius/round semantics

Code (latent-cpu/src/lib.rs:65-74, 234-262): two 1-D passes, each averaging $2r + 1$ taps with `clamp(0, dim-1)` indices; `r = radius.round().max(0)`.

Standard. A 2-D box kernel is the outer product of two 1-D box kernels, so $B_r^{2D} = B_r^{\text{vert}} * B_r^{\text{horiz}}$ exactly — the separation is an $O(r)$ -per-pixel optimization with **no** approximation. Verified by the test `blur_matches_a_box_average_reference` (lib.rs:408). Border handling is **clamp-to-edge** (edge-pixel replication), one of the standard pad modes (Wikipedia, *Box blur*, “Extend / Fill in a constant color extending from the last pixel”); it is energy-preserving (the divisor stays $2r + 1$, and replicated samples are real edge values), so a uniform image is unchanged (test `blur_leaves_a_uniform_image_unchanged`).

VERDICT: Correct. Separable box blur with clamp-to-edge is a standard, exact, energy-preserving implementation. **Citation:** Wikipedia, *Box blur* (separability; border-fill options).

Radius/round consistency (see §3, Finding M3). `blur` uses `r = radius.round().max(0)` (so radius $0 \rightarrow$ identity, radius $<0.5 \rightarrow$ identity); `bilateral_pixel` uses `r = radius.round().max(1)` (window $\pm r$); `dehaze` uses a **constant** `DEHAZE_PATCH=4` independent of any radius. The blur/denoise rounding is *consistent with each other* (both round), but the gate predicates differ slightly: `apply_global` gates blur on radius > 0.0 (so radius 0.3 rounds to $0 \rightarrow$ silent no-op clone) while denoise gates on `params.radius.round() >= 1.0`. Harmless, but the dehaze patch size being a fixed constant (not tied to the UI “radius” the other tools expose) is an inconsistency worth a doc note (§3, M2).

4.2.3 2.3 Clarity — three box blurs as a Gaussian, and midtone weighting

4.2.3.1 2.3.a Three box blurs $\approx$ Gaussian (central limit)

Claim (lib.rs:421-423): three box passes are “a central-limit approximation of a Gaussian — because a single box kernel rings at the broad clarity radius and would itself create halos.”

Standard. By the central limit theorem, repeated box convolution converges to a Gaussian (Wikipedia, *Box blur*: “By the central limit theorem, repeated application of a box blur will approximate a Gaussian blur”, citing Getreuer 2013). Three passes is the **textbook count** for a fast Gaussian approximation (Wells 1986, *Efficient synthesis of Gaussian filters by cascaded uniform filters*, PAMI 8(2):234–239 — the canonical 3-to-4-pass box-cascade result; widely repeated, e.g. the CSS/SVG blur spec uses a 3-box approximation).

Variance / effective σ . A single discrete box mean over integer offsets $[-r, r]$ has variance $\sigma_1^2 = \frac{(2r+1)^2-1}{12} = \frac{r(r+1)}{3}$. Independent passes **add** variance, so K boxes give $\sigma_K^2 = K \cdot \frac{r(r+1)}{3}$. For $K = 3$:

$$\sigma_{3\text{box}} = \sqrt{3 \cdot \frac{r(r+1)}{3}} = \sqrt{r(r+1)} \xrightarrow{r \rightarrow \infty} r.$$

Evidence (Python, math only):

```
== 3-box effective sigma ==
r= 1  sigma_3box=  1.414  s3/r=1.4142
r= 3  sigma_3box=  3.464  s3/r=1.1547
r= 5  sigma_3box=  5.477  s3/r=1.0954
r=10  sigma_3box= 10.488  s3/r=1.0488
r=50  sigma_3box= 50.498  s3/r=1.0100  # -> r
```

So **three boxes of radius r give an effective Gaussian $\sigma \approx r$** (exactly $\sqrt{r(r+1)}$, within 1% of r for $r \geq 10$). Useful, sensible scaling for a “clarity radius”.

How close to a true Gaussian? Kernel-shape comparison of K -box cascades against a same-variance sampled Gaussian (max-abs and L_1 error over the kernel):

```
r= 5 K=2: L1=0.0937    r= 5 K=3: L1=0.0509    r= 5 K=4: L1=0.0381
r=10 K=2: L1=0.0932    r=10 K=3: L1=0.0510    r=10 K=4: L1=0.0381
excess kurtosis of K-box -> 0: K=1:-1.200 K=2:-0.600 K=3:-0.400 K=4:-0.300
```

Three boxes give $\sim 5\%$ L_1 kernel error and excess kurtosis -0.4 (a single box is -1.2). The bulk of the improvement is $1 \rightarrow 3$; $3 \rightarrow 4$ only trims ~ 1.3 pts more L_1 . **Three is the right, conventional count** — a good cost/quality knee. The motivation in the comment (a single box rings/halos at a broad radius) is exactly right: a box kernel has a sinc frequency response with large side-lobes; cascading suppresses them.

VERDICT: Correct. Three-box Gaussian approximation is standard practice and the effective $\sigma \approx r$ is a sound, documented relationship. **Citations:** Wells, PAMI 1986 (cascaded uniform filters $\approx$ Gaussian, 3–4 passes); Wikipedia, *Box blur* (CLT statement, Getreuer 2013); Getreuer, *A Survey of Gaussian Convolution Algorithms*, IPOL 2013.

4.2.3.2 2.3.b Midtone parabola placement

Code (lib.rs:453-456): $m(\ell) = 1 - (2b - 1)^2$, $b = \text{clamp}(\ell)^{1/2.2}$, evaluated on the **base (low-frequency) luminance**.

Claim (lib.rs:445-452): the window peaks on perceptual mid-gray (≈ 0.18 linear) rather than linear 0.5, “genuinely weights the midtones,” and protects high-lights/shadows from halos.

Numeric check (Python):

```
Parabola peaks (m=1) at b=0.5 => linear luma = 0.5^2.2 = 0.2176
m(0.18) = 0.9932    (photographic mid-gray ~0.18 linear)
m(0.5)  = 0.7889    (linear 0.5 sits well off-peak, as intended)
half-max (m=0.5) at linear luma = 0.0146 and 0.7058
L*=50 mid-gray linear = 0.1842
```

The peak lands at **linear 0.2176** ($= 0.5^{2.2}$), within striking distance of the photographic 18% gray card (0.18) and the perceptual $L^* = 50$ mid-gray (0.184). $m(0.18) = 0.993$ — essentially full weight at mid-gray — while linear 0.5 is pushed down to $m = 0.789$ and the extremes go to 0. The claim holds quantitatively. Evaluating m on the **blurred base** (not the high-frequency detail) is the right choice: it’s the local *tone* that should gate the contrast boost, and it makes the gate spatially smooth so the protection doesn’t itself create a hard transition. The half-max points (linear 0.015 and 0.706) show the window genuinely tapers contrast toward both black and white, which is what suppresses clarity halos at the tonal extremes.

VERDICT: Correct. The parabola peaks on perceptual mid-gray as claimed; midtone-gating the local-contrast term is a sound, halo-aware design. The `clarity_boosts_midtone_local_contrast` and `local_contrast_amplifies_midtones_and_protects_the_extremes` tests confirm the behavior. **Citation:** Photographic 18%-gray / $L^* = 50$ mid-gray conventions (CIE L^* , ISO 7589 / standard middle-gray); the parabola $1 - (2b - 1)^2$ is the unique downward parabola with zeros at $b \in \{0, 1\}$ and peak 1 at $b = 0.5$.

4.2.4 2.4 Bilateral denoise — vs Tomasi & Manduchi (ICCV 1998)

Reference equations (docs/spatial-bilateral-tomasi-1998.pdf). The combined bilateral filter (their eqs. 5-6) averages each pixel with weights that are the **product** of a domain (spatial) Gaussian and a range (photometric) Gaussian, normalized by the weight sum:

$$h(x) = k^{-1}(x) \int f(\xi) \underbrace{c(\xi, x)}_{\text{domain}} \underbrace{s(f(\xi), f(x))}_{\text{range}} d\xi, \quad k(x) = \int c s d\xi,$$

with the Gaussian case $c = e^{-\frac{1}{2}(d/\sigma_d)^2}$, $s = e^{-\frac{1}{2}(\delta/\sigma_r)^2}$, $d = \|\xi - x\|$ (geometric spread σ_d), $\delta = \|f(\xi) - f(x)\|$ (photometric spread σ_r).

Code (`bilateral_pixel`, lib.rs:525-568). Spatial term `spatial = -(dx2+dy2)·inv_2ss2`, range terms subtracted in the exponent: `wl = exp(spatial - dl2·inv_2sl2)`, normalized by `wsum`. This is **exactly** $e^{-\frac{\|d\|^2}{2\sigma_s^2}} e^{-\frac{\delta^2}{2\sigma_r^2}}$ as a single `exp(sum)` — algebraically identical to the product of the two Gaussians, normalized by the weight sum. ✓

Point-by-point:

Aspect	Paper	Code	Verdict
Weight = domain $\times$ range	eq. 5–6, product of two Gaussians	<code>exp(spatial - $\Delta^2 \cdot \text{inv}2\sigma^2$) = product</code>	Correct
Normalization by $k = \sum w$	eq. 6	<code>acc/wsum per component</code>	Correct
Spatial Gaussian	σ_d “based on desired low-pass”	$\sigma_s = r/2$	Correct-with-caveats (truncation, below)
Range Gaussian	σ_r photometric spread	σ_l, σ_c	Correct form
Edge preservation	range term rejects cross-edge pixels	same	Correct (test ... <code>keeps_an_edge</code>)
Color handling	single combined distance, ideally CIE-Lab (§5)	two separate Y/chroma filters	Correct-with-caveats (variant, below)

Caveat 1 — luma/chroma split vs the paper’s combined distance (Finding M1).

Tomasi & Manduchi §5 are explicit that for color one should **not** filter R,G,B separately — “*Separate smoothing results in an even more pronounced pink-purple band*” — but instead “*combine the three color bands appropriately, and measuring photometric distances between pixels in the combined space ... using Euclidean distance in the CIE-Lab color space ... the most natural type of filtering for color images.*” i.e. **one** range weight from a single perceptual color distance.

The code instead runs **two** bilateral passes with **two** range weights: luma (scalar ΔY) and chroma (vector ΔC in an RGB-minus-luma offset space). This is *not* the per-RGB-channel mistake the paper warns against (that would be three independent scalar filters); it is the **standard luma/chroma denoising decomposition** used throughout raw/photo NR (Lightroom/ACR, dcrw/wavelet NR, “Luminance” + “Color” sliders), and it has a real benefit the paper’s single filter lacks: chroma can be smoothed **harder** than luma without destroying luminance detail, because color noise is low-frequency blotch. It is therefore a **legitimate, well-established variant** (close in spirit to YCbCr / cross-component bilateral denoising), but it **departs from the cited paper’s formulation** and is not what T&M eq. 5–6 describes. The chroma distance is measured in a linear RGB-offset space, not CIE-Lab, so it is *not* perceptually uniform — a moderate accuracy point, not an error. **Recommendation:** keep the split (it’s the right tool), but cite it as a luma/chroma variant rather than implying it is the T&M color filter, and consider that the chroma metric is non-perceptual.

Caveat 2 — spatial truncation at $\pm 2\sigma$ (Finding H1, the flagged item). With $\sigma_s = r/2$ and window $\pm r$, the window edge is at exactly 2σ . The spatial weight there is $e^{-2} = 0.135$ — a *hard* cutoff at 13.5% of peak, not the near-zero a properly supported Gaussian reaches.

Spatial Gaussian at window edge (2σ): `exp(-2) = 0.1353`

At 3σ it would be: $\exp(-4.5) = 0.0111$
 1D Gaussian mass beyond $\pm 2\sigma = \text{erfc}(2/\sqrt{2}) = 0.0455$ ($\sim 4.6\%$ per axis)
 1D Gaussian mass beyond $\pm 3\sigma = \text{erfc}(3/\sqrt{2}) = 0.0027$ ($\sim 0.27\%$)

The standard convention is to truncate a Gaussian at 3σ (SciPy `gaussian_filter` defaults `truncate=4.0`; common kernel size $2\lceil 3\sigma \rceil + 1$; Bart Wronski / general CV practice: radius $2\text{--}3\sigma$). Truncating at 2σ drops $\sim 4.6\%$ of the 1-D Gaussian mass per axis ($\sim 9\%$ in 2-D) and, more importantly, leaves a **discontinuous step** in the kernel at the window boundary (weight jumps from 0.135 to 0). For an edge-preserving filter this is mostly cosmetic (the range term already gates most far pixels), but it (a) makes the spatial kernel a truncated-Gaussian-with-a-step rather than a smooth one, and (b) means σ_s is effectively smaller than the window suggests. **Recommendation:** either set $\sigma_s = r/3$ (window $\pm 3\sigma$, the standard), or keep $\sigma_s = r/2$ but extend the window to $\pm \lceil 3\sigma_s \rceil = \pm \lceil 1.5r \rceil$. The current $\pm 2\sigma$ is *defensible* (it bounds cost and the comment acknowledges the choice — “falls off across the support ... rather than behaving like a box”) but is **narrower support than the literature standard** and should be flagged.

Other observations. - **Luma weights (ProPhoto-D65).** LUMA_WEIGHTS = [0.2788, 0.7211, 0.000113] — the blue weight is $\sim 10^{-4}$ (color.rs:177-181 flags this as colorimetrically correct for wide primaries). Consequence for the chroma split: blue contributes essentially **nothing** to Y , so the blue chroma offset $C_b = B - Y \approx B$ carries almost the full blue signal. The chroma filter therefore does the heavy lifting on blue. That’s internally consistent (luma+chroma still reconstruct RGB exactly: $O = Y_{\text{out}} + C_{\text{out}}$ telescopes back), but it means blue **luminance detail** is effectively governed by the *chroma* scale, not the luma scale — a subtle consequence worth a doc note (Finding L1). - **Non-separability / performance (Note N2).** The bilateral is genuinely non-separable (the range weight depends on 2-D content), so it is $O(r^2)$ per pixel — correct and unavoidable for the exact filter. The code parallelizes rows via Rayon. No fast-bilateral approximation (bilateral grid, permutohedral) is used; fine for a reference backend, a performance note for large radii. - **Border.** `clamp(0, dim-1)` index replication, consistent with the box blur. Correct.

VERDICT (overall): Correct-with-caveats. The core bilateral math (product of spatial $\times$ range Gaussians, normalized by weight sum, edge-stopping) matches Tomasi & Manduchi exactly. The two departures are (i) the luma/chroma split with separate range weights — a standard denoising variant, but **not** the paper’s single-combined-distance color filter, and in a non-perceptual chroma metric; and (ii) the $\pm 2\sigma$ spatial truncation vs the standard 3σ . **Citation:** Tomasi & Manduchi, ICCV 1998, eqs. 1-6 (domain/range/combined) and §5 (color: combined distance, CIE-Lab, against per-channel filtering).

4.2.5 2.5 Dehaze — vs He, Sun & Tang (CVPR 2009 / TPAMI 2011)

Reference equations (docs/spatial-dehaze-he-2011-tpami.pdf): - **Model** eq. (1): $I(x) = J(x)t(x) + A(1 - t(x))$. $\checkmark$ matches code (`dehaze_recover` doc + impl). - **Dark channel** eq. (5): $J^{\text{dark}}(x) = \min_{y \in \Omega(x)} (\min_c J^c(y))$. $\checkmark$ matches `dehaze_dark_channel` (patch-min of channel-min). - **Transmission** eq. (12):

$\tilde{t}(x) = 1 - \omega \min_y (\min_c \frac{I^c(y)}{A^c})$, with $\omega = 0.95$. The code computes $t = 1 - \text{strength} \cdot \text{dc}$ with $\text{dc} = \min_y \min_c I^c$ and $A = 1$, so strength is ω and $\text{dc} = \min_y \min_c I^c / A^c$ holds **only because** $A = 1$. ✓ (conditional on $A = 1$). - **Lower bound** $t_0 = 0.1$ (eq. 22 text: “A typical value of t_0 is 0.1”). ✓ $\text{DEHAZE_T0}=0.1$. - **Recovery** eq. (22): $J(x) = \frac{I(x)-A}{\max(t(x), t_0)} + A$. ✓ matches $\text{dehaze_recover}(\text{in_range}-1)/t + 1$ with $A = 1$ and t pre-clamped to $[t_0, 1]$. - **Patch size**: He uses 15×15 (§4.5, “in the remainder of this paper, we use a patch size of 15×15 ”). Code uses $\text{DEHAZE_PATCH}=4 \rightarrow 9 \times 9$.

So the **model, prior, transmission form, t_0 floor, and recovery equation are transcribed correctly**. Two deliberate simplifications and one parameter difference:

Simplification 1 — fixed airlight $A = 1$ instead of estimating A (Finding H2, flagged). He §4.3 estimates A from the image: “We first pick the top 0.1 percent brightest pixels in the dark channel ... Among these pixels, the pixels with highest intensity in the input image I are selected as the atmospheric light.” This per-channel $A = (A^r, A^g, A^b)$ is the **key step** that lets the method (a) handle **non-neutral / colored haze** (warm sunset haze, blue distance haze) by normalizing each channel by its own A^c in eq. (7), and (b) place the true white point so recovered colors are correct.

Fixing $A = 1$ (neutral white, at the working-space ceiling) means: - **Color cast in the veil is not removed** — only a neutral gray veil is correctly inverted. Real haze is often slightly blue or warm; with $A = 1$ that tint is *baked into* J (the recovery $J = (I - 1)/t + 1$ shifts all channels by the **same** t , so a colored veil stays colored). The `dehaze_clears_a_synthetic_veil` test (`lib.rs:1176`) only validates the **white-airlight** case — exactly the case where $A = 1$ is correct — so it does not exercise this gap. - **Magnitude assumption**: with linear-light headroom, scene values can exceed 1 and a true diffuse-white airlight may be **brighter than 1** (a bright sky), so $A = 1$ can *under-estimate* A , under-removing haze; or the brightest haze may be **below 1**, so $A = 1$ *over-estimates*, pushing recovered darks too far down. The code’s headroom passthrough ($I > 1$ left unchanged) is a reasonable guard but it means the densest/brightest haze region is *not* dehazed at all. - **Upside**: it removes the fragile, content-dependent A estimation (which can be fooled by white objects — the very failure He §4.3 designs around) and is deterministic/local, which suits a per-pixel GPU-friendly primitive. For a *neutral* veil it is exactly correct.

Impact: Medium-High. For neutral haze the result matches He; for **colored** haze the veil’s tint is not neutralized, and for scenes where true airlight $\neq 1$ the strength is mis-scaled. **Recommendation**: estimate A (even a cheap global estimate: per-channel mean/percentile of the brightest dark-channel pixels) and normalize I/A per channel before the dark-channel and recovery, per He eqs. (7),(12),(22). At minimum, document that dehaze assumes a neutral airlight and is only colorimetrically correct for gray veils.

Simplification 2 — raw patch transmission, no soft-matting / guided-filter refinement (Finding H3, flagged). He §4.2 refines $\tilde{t}$ with a soft-matting Laplacian (and, in their later work, the guided filter) because “the transmission is not always constant in a patch ... The main problems are some halos and block artifacts” (§4.1, Fig. 6). The code uses the **raw patch transmission** directly. Consequence: **block / halo artifacts at depth edges**, exactly as He Fig. 6(b)/Fig. 10(c) show for the un-refined map. The per-pixel patch-min already softens this slightly (each pixel re-

computes its own 9×9 min, so t varies per pixel rather than per block), which is **better than a true block transmission** but still unrefined — transmission will be over-flat across depth discontinuities, producing the characteristic dark halo around foreground edges against bright haze.

Impact: Medium. Visible as edge halos/blocking at strong settings. **Recommendation:** refine t with a guided filter (He & Sun, *Guided Image Filtering*, ECCV 2010 — their own faster replacement for soft matting), using the input luminance as guide. This is the standard modern fix and is $O(N)$.

Parameter — 9×9 patch vs He’s 15×15 (Finding M2). He §4.5 analyzes patch size: too small $\rightarrow$ oversaturated recovery (dark channel not dark enough, eq. 9 violated); too large $\rightarrow$ stronger halos. They settle on 15×15 for ~ 500 – 600 px images. `DEHAZE_PATCH=4` (9×9) is **smaller**, biasing toward He’s “oversaturated” regime (the dark channel is less likely to hit a true dark pixel, so dc is over-estimated, t under-estimated, haze *over-removed* and colors *over-saturated*). It is also a **fixed absolute size**, not scaled to image resolution — on a 24MP raw, 9×9 is a much smaller fraction of the frame than 15×15 was on He’s 0.3MP test images, pushing further into the small-patch regime. **Recommendation:** scale the patch with resolution (or expose it), and consider $\geq 15 \times 15$ -equivalent at He’s reference scale to avoid over-saturation; the patch-min already partially compensates by sparing bright neutral subjects (the `dehaze_preserves_a_bright_neutral_subject` test), but the absolute size is small.

VERDICT (overall): Correct-with-caveats. The scattering model, dark-channel prior, transmission, t_0 floor, and recovery equation are correct transcriptions of He et al. The implementation deliberately drops the two refinements that make the published method robust on real images: **airlight estimation** (fixed $A = 1$) and **transmission refinement** (raw patch t), and uses a **smaller-than-recommended, fixed patch**. These are acceptable simplifications for a fast neutral-veil dehaze but should be documented and ideally addressed. **Citations:** He, Sun & Tang, TPAMI 2011 / CVPR 2009: eq. (1) model, eq. (5) dark channel, eq. (12) transmission ($\omega = 0.95$), §4.3 airlight (top-0.1% dark-channel pixels), eq. (22) recovery ($t_0 = 0.1$), §4.2 soft matting, §4.5 patch size 15×15 . Refinement: He & Sun, *Guided Image Filtering*, ECCV 2010.

4.2.6 2.6 Ordering and resolution (denoise $\rightarrow$ dehaze $\rightarrow$ clarity $\rightarrow$ sharpen, before resample)

Code (`apply_global`, `lib.rs:395-441`; geometry runs later in `apply_geometry`, `lib.rs:689`).

Denoise before sharpen — correct. Sharpening amplifies high frequencies, including noise; denoising first prevents the unsharp/clarity stages from boosting the very grain the bilateral removed. The comment at `lib.rs:399` states this rationale, and it is the universally recommended order (denoise $\rightarrow$ enhance). ✓

Dehaze before clarity/sharpen — reasonable. Dehaze is a tone/contrast restoration that changes the global contrast envelope; doing local-contrast (clarity) and edge

sharpening *after* it operates on the already-restored signal, which is the sensible order (you sharpen what you intend to keep).

Doing all of this at full source resolution, then resampling later (Finding M4 / Note). Geometry (rotation, perspective, distortion, scale) resamples **after** all spatial filters. Two consequences: - **Sharpening then downscaling:** if the geometry stage *downscales* (e.g. fitting to a smaller output, or strong perspective compression), the sharpening was tuned at source resolution and then resampled. Downscaling sharpened content can **alias the overshoots** and waste the sharpening (the high-frequency detail you boosted is partly thrown away or moiré'd by the bilinear resampler, which has no prefilter — see Audit on geometry). The textbook order for output sharpening is **resample first, then sharpen at output resolution** (“output sharpening” / capture-vs-output sharpening, the standard raw-workflow distinction). Running sharpen at source res is “capture sharpening,” which is defensible, but if the output is significantly smaller it is suboptimal. - **Upscaling:** if geometry upscales, sharpening at source res is fine or even preferable (sharpen the real detail before interpolation invents none). - **Filter radii are in source pixels:** a clarity/denoise radius chosen at source resolution will not correspond to the same *output* feature size after a scale change. For a fixed-pipeline developer this is usually acceptable (the user tunes against the source), but it means “radius” is resolution-dependent.

VERDICT: Correct (ordering) / Note (resolution). The intra-stage order is correct (denoise first, sharpen last). Performing spatial filters at source resolution before geometry is **capture-sharpening** semantics — correct when geometry preserves or increases scale, but suboptimal when geometry **downscales** (overshoots get resampled by an unfiltered bilinear sampler, risking aliasing and wasted sharpening). Worth a design note; not a correctness bug for the upscale/no-scale common case. **Citation:** Standard raw-workflow capture-vs-output sharpening (e.g. Bruce Fraser, *Real World Image Sharpening*); NVIDIA GPU Gems 3 Ch. 24 (filtering correctness under resampling).

4.3 3. Findings by Severity

ID	Sev	File:line	Issue	Reference	Recommendation
H1	High	latent- pipeline/src/lib 529	Bilateral spatial Gaus- sian trun- cated at $\pm 2\sigma$ ($\sigma_s =$ $r/2$, window $\pm r$). Edge weight $e^{-2} =$ 0.135 (hard step); $\sim 4.6\%$ /axis of Gaussian mass dropped; effective support narrower than the kernel implies.	Standard 3σ truncation (SciPy truncate=4.0; kernel $2[3\sigma] + 1$).	Use $\sigma_s = r/3$ (window = $\pm 3\sigma$) or keep $\sigma_s = r/2$ and widen window to $\pm[1.5r]$.

ID	Sev	File:line	Issue	Reference	Recommendation
H2	High	latent-pipeline/src/lib/latent/latent_pipeline.cpp:505	Dehaze fixes 493- airlight $A = 1$ instead of esti- mating per- channel A . Colored haze is not neu- tralized; strength mis- scaled when true airlight $\neq 1$. Only correct for a neutral, ≤ 1 veil.	He et al. TPAMI 2011, §4.3 (top-0.1% dark-channel pixels $\rightarrow$ highest- $I \rightarrow A$); eqs. (7),(12),(22) use I/A .	Estimate A ($\geq$ a cheap global per-channel percentile of brightest dark-channel pixels) and normalize I/A^c per channel; or document the neutral-veil limitation.
H3	High	latent-pipeline/src/lib/latent/latent_pipeline.cpp:487	No trans- 475- mission refine- ment — raw per-pixel patch $t \rightarrow$ block/halo artifacts at depth edges.	He et al. §4.2 (soft matting) / He & Sun, <i>Guided Image Filtering</i> , ECCV 2010.	Refine t with an $O(N)$ guided filter (guide = input luma) before recovery.

ID	Sev	File:line	Issue	Reference	Recommendation
M1	Medium	latent-pipeline/src/lib.rs:525-567	Denoise luma/chroma split with two separate range weights , not T&M's single combined-distance (CIE-Lab) color filter; chroma distance is in a non-perceptual linear-RGB-offset space.	Tomasi & Manduchi ICCV 1998 §5 (combine bands, CIE-Lab; per-channel filtering is worse).	Keep the split (standard NR variant, intentional luma≠chroma strength), but document it as a luma/chroma variant; consider a perceptual chroma metric.
M2	Medium	latent-pipeline/src/lib.rs:468	Dehaze patch 9×9 (DEHAZE_PATCH=4) fixed absolute size , vs He's 15×15 ; biases toward the small-patch <i>over-saturation</i> regime, worse on high-MP rasters.	He et al. §4.5 (patch-size analysis; 15×15 at ~ 0.3 - 0.4 MP).	Scale patch with resolution (or expose it); target $\geq$ He's 15×15 -equivalent at reference scale.

ID	Sev	File:line	Issue	Reference	Recommendation
M4	Medium	latent-pipeline/src/lib.rs:355-357	Spatial filter ³⁵⁵ -run at source resolution before geometry ; if geometry down-scales , sharpening overshoots are re-sampled by an unpre-filtered bilinear sampler → aliasing/wasted sharpening.	Capture-vs-output sharpening (Fraser); GPU Gems 3 Ch. 24.	Consider an output-sharpening pass after resample, or prefilter on downscale. (Fine for no-scale/upscale.)
L1	Low	latent-image/src/color.rs:535-536	ProPhoto-D51 blue luma weight $\approx 10^{-4}$, so blue's luminance detail is governed by the chroma scale, not luma, in the denoise split.	(consequence of wide-gamut luminance)	Document; consider clamping blue weight or a luma floor if blue detail is over-smoothed.

ID	Sev	File:line	Issue	Reference	Recommendation
N1	Note	latent-pipeline/src/libsrc/latent-440	Sharpening on linear-light: physically correct for blur, but unsharp over-shoot is asymmetric perceptually (brighter dark-side halos) vs gamma/luma-domain sharpeners; sharpens all RGB (potential edge color fringing) rather than luma-only.	NVIDIA GPU Gems 3 Ch. 24; Wikipedia/Imatest (over-shoot→halos).	Acceptable given linear-light philosophy; document. Optionally sharpen luma only.
N2	Note	latent-pipeline/src/libsrc/latent-567	Bilateral and non-separable → $O(r^2)$ /pixel; no fast-bilateral approximation.	Tomasi & Manduchi (non-separable).	Fine as reference; consider bilateral-grid/permutohedral for large radii.

No Critical findings. Every formula transcribed (unsharp, box, midtone parabola, bilateral product-of-Gaussians, dehaze model/transmission/recovery) matches its au-

thoritative source; the issues are deliberate simplifications and support/parameter choices, not transcription errors.

4.4 4. Verdict Summary

Filter	Verdict
Box blur (separable, clamp border)	Correct
Unsharp mask (gain=1+amount)	Correct (linear-domain caveat N1)
Clarity: 3-box Gaussian approx ($\sigma \approx r$)	Correct
Clarity: midtone parabola (peak at linear 0.2176)	Correct
Bilateral denoise (core math)	Correct
Bilateral: luma/chroma split + 2σ support	Correct-with-caveats (M1, H1)
Dehaze: model / DCP / transmission / t_0 / recovery	Correct
Dehaze: fixed $A = 1$, no t -refinement, 9×9 patch	Correct-with-caveats (H2, H3, M2)
Ordering (denoise→dehaze→clarity→sharpen)	Correct
Full-res then resample	Note (M4)

4.5 5. References

1. C. Tomasi, R. Manduchi, **Bilateral Filtering for Gray and Color Images**, *Proc. IEEE ICCV 1998*, pp. 839–846. Local copy: docs/spatial-bilateral-tomasi-1998.pdf. <https://users.soe.ucsc.edu/~manduchi/Papers/ICCV98.pdf>
2. K. He, J. Sun, X. Tang, **Single Image Haze Removal Using Dark Channel Prior**, *IEEE TPAMI 33(12):2341–2353, 2011* (extended version of *CVPR 2009*, pp. 1956–1963). Local copy: docs/spatial-dehaze-he-2011-tpami.pdf. <http://mmlab.ie.cuhk.edu.hk/2011/Haze.pdf>
3. IPOL, **Dehazing with Dark Channel Prior** (peer-reviewed reproduction), *IPOL 2024*. Local copy: docs/spatial-dehaze-ipol-2024.pdf. <http://www.ipol.im/pub/art/2024/530/>
4. K. He, J. Sun, X. Tang, **Guided Image Filtering**, *ECCV 2010* (authors’ $O(N)$ replacement for soft matting in transmission refinement). <https://kaiminghe.github.io/eccv10/>
5. W. M. Wells, **Efficient Synthesis of Gaussian Filters by Cascaded Uniform Filters**, *IEEE TPAMI 8(2):234–239, 1986* (box-cascade $\approx$ Gaussian; 3–4 passes).
6. P. Getreuer, **A Survey of Gaussian Convolution Algorithms**, *IPOL 2013* (box-blur CLT / repeated-box Gaussian). <https://www.ipol.im/pub/art/2013/87/>

7. Wikipedia, **Unsharp masking** — sharpened = original + (original − blurred) × amount; overshoot/halo. https://en.wikipedia.org/wiki/Unsharp_masking
8. Wikipedia, **Box blur** — separability, CLT Gaussian approximation, border-fill modes. https://en.wikipedia.org/wiki/Box_blur
9. Imatest, **Sharpening** — USM equation $L_{\text{USM}} = L - k_{\text{USM}} \text{Blur}$; “small overshoots enhance sharpness; large overshoots cause halos.” <https://www.imatest.com/imagining/sharpening/>
10. L. Gritz, E. d’Eon, **The Importance of Being Linear**, *GPU Gems 3*, Ch. 24, NVIDIA — filtering/blending/mip must be in linear light. <https://developer.nvidia.com/gpugems/gpugems3/part-iv-image-effects/chapter-24-importance-being-linear>
11. SciPy, **scipy.ndimage.gaussian_filter** — truncate=4.0 default (radius $\approx 4\sigma$; 3σ minimum convention). https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.gaussian_filter.html
12. B. Fraser, J. Schewe, **Real World Image Sharpening with Adobe Photoshop, Camera Raw, and Lightroom** — capture vs. output sharpening (sharpen at output resolution).

4.5.1 Appendix A — Numeric evidence (reproducible, python3, stdlib math only)

3-box effective σ ($\sigma_{3\text{box}} = \sqrt{r(r+1)} \rightarrow r$): $r=3 \rightarrow 3.464$, $r=10 \rightarrow 10.488$, $r=50 \rightarrow 50.498$.

3-box vs true Gaussian (kernel L_1 error): $K=2:0.094$, $K=3:0.051$, $K=4:0.038$; excess kurtosis $K=1:-1.2 \rightarrow K=3:-0.4$. Three is the cost/quality knee.

Midtone parabola peak: m peaks ($=1$) at $b=0.5 \Rightarrow$ linear luma = $0.5^{2.2} = 0.2176$; $m(0.18) = 0.993$, $m(0.5) = 0.789$; half-max at linear 0.015 and 0.706. ($L^*=50$ mid-gray = 0.184.)

Bilateral 2σ truncation: spatial weight at window edge = $e^{-2} = 0.135$; 1-D Gaussian mass beyond $\pm 2\sigma = 4.6\%$, beyond $\pm 3\sigma = 0.27\%$.

Unsharp algebra: $L + (1 + a)(I - L) \equiv I + a(I - L)$, verified to $< 10^{-9}$ on random inputs.

Chapter 5

Audit 05 — Geometry & Optics

Component: latent-pipeline (geometry stage), latent-cpu (resampler), latent-gpu (GPU resampler), latent-lens (lensfun → profile mapping) **Files audited:** latent-pipeline/src/lib.rs, latent-cpu/src/lib.rs, latent-gpu/src/resample.wgsl, latent-lens/src/lib.rs (with cross-reference to latent-edit/src/lib.rs for LensProfile). **Scope:** the single SOURCE→OUTPUT geometry stage — radial lens distortion, lateral chromatic aberration (TCA), keystone (perspective) homography, straighten (rotation) with bounding-box expansion, the single bilinear resample, crop, lens-vignetting correction (pre-resample flat-field), and the creative vignette (post-crop gain); plus the lensfun coefficient → LensProfile conventions. **Date:** 2026-06-27 **Method:** Each source file read line-by-line, then every model/coefficient convention verified against primary sources: lensfun’s lfDistortionModel/lfTCAModel/lfVignettingModel API docs, the lensfun “How the corrections work” page, and the **lensfun C++ source** (libs/lensfun/modifier.cpp, libs/lensfun/mod-coord.cpp) for the exact NormScale normalization and the forward-vs-backward direction; the PanoTools/Hugin lens-correction-model wiki for the “abc” model; and Szeliski, *Computer Vision: Algorithms and Applications* (§3.6) for resampling/warping best practice. Reference docs are downloaded under docs/ with a geometry- prefix.

5.1 1. Summary of the Geometry Model and the Single-Resample Design

latent folds every geometric correction into **one** coordinate map and interpolates **exactly once**. The map is OUTPUT→SOURCE (inverse mapping). In apply_geometry (latent-pipeline/src/lib.rs:689) the order is:

1. **Lens vignetting correction** — a SOURCE-space radial-gain *multiply* (RadialGain, reciprocal = true) applied **before** any resample (matching lensfun’s vignetting→geometry order). A flat-field, not an interpolation.
2. **Build one homography** = keystone ◦ straighten (matrix product via Transform::compose, output canvas = straighten’s expanded bounding box).

3. **Resample once** through a Warp: homography (with perspective divide), then a radial distortion $s(r) = 1 + d_0 r + d_1 r^2 + d_2 r^3 + d_3 r^4$ about the optical center, then a per-channel radial scale for TCA. If there is no lens term it degrades to a plain homography resample. Bilinear, in linear light.
4. **Crop** — an exact integer clip of the resampled result.
5. **Creative vignette** — a radial-gain *multiply* about the post-crop center, corners normalized to $r = 1$.

The single-resample principle is the headline design strength and is correct (Szeliski §3.6.1: composing transforms and resampling once avoids the blur of repeated interpolation). The resampler is a 2-tap bilinear with no prefilter; the geometry stage is intended for $\sim$ unit-scale maps (crop/rotate/mild distortion), with downscaling handled elsewhere by area-averaging (ImageBuf::downscaled, per the sample_bilinear comment at latent-cpu/src/lib.rs:264).

The single most important conclusion of this audit: the *direction* (forward formula used as a backward map) is defensible and matches the lensfun convention, and the *vignetting* and *TCA* conventions match; **but the distortion/vignetting/TCA radius normalization unit does not match lensfun's**. The code normalizes by **half the shorter side** ($\text{inv_norm} = 2 / \min(w, h)$); lensfun normalizes by the **half-diagonal** (a focal-length-scaled 35 mm-equivalent unit that, for the common case, puts $r \approx 1$ at the image *corner*, not the short edge). Feeding real lensfun coefficients through the current normalization silently rescales every distortion/TCA/vignetting coefficient. This is the highest-risk correctness gap.

5.2 2. Point-by-Point Verification

5.2.1 2.1 Radial distortion model & coefficient form — Warp::map, radial_distortion

Code (latent-pipeline/src/lib.rs:264-267):

```
// s(r) = 1 + d0·r + d1·r² + d2·r³ + d3·r⁴ (Horner in r).
let [d0, d1, d2, d3] = self.radial;
let s = 1.0 + r * (d0 + r * (d1 + r * (d2 + r * d3)));
(self.center[0] + dx * s, self.center[1] + dy * s)
```

Applied as $p' = c + (p - c) s(r)$, i.e. $r_{\text{src}} = r s(r)$.

Authoritative models (lensfun lfDistortionModel, docs/geometry-lensfun-lens-models.html): - **POLY3**: $r_d = r_u(1 - k_1 + k_1 r_u^2)$. - **POLY5**: $r_d = r_u(1 + k_1 r_u^2 + k_2 r_u^4)$. - **PTLENS** (PanoTools/Hugin): $r_d = r_u(a r_u^3 + b r_u^2 + c r_u + 1 - a - b - c)$.

- (a) **Brown-Conrady radial is even-powered** — confirmed. Brown's radial distortion is $r(1 + k_1 r^2 + k_2 r^4 + \dots)$; lensfun's POLY3/POLY5 are exactly this even subset. (b) **PanoTools is the cubic abc form** — confirmed: PanoTools/Hugin "Lens correction model" gives $r_{\text{src}} = (a r_d^3 + b r_d^2 + c r_d + d) r_d$ with $d = 1 - (a + b + c)$ "to keep the same image size" (docs/geometry-panotools-lens-correction-model.html). The code's general $1 + d_0 r + d_1 r^2 + d_2 r^3 + d_3 r^4$ (note: d_0, d_2

odd, d_1, d_3 even) is a strict superset that can express all three; the even-only subset is Brown-Conrady and the odd terms enable PTLens. The mapping in `radial_distortion` (`latent-lens/src/lib.rs:183`) is algebraically correct:

- $\text{POLY5} \rightarrow [0, k_1, 0, k_2]$ (exact).
- $\text{POLY3} \rightarrow [0, k_1/(1 - k_1), 0, 0]$ — factors out POLY3’s built-in $(1 - k_1)$ corner-anchor magnification.
- $\text{PTLENS} \rightarrow [c/D, b/D, a/D, 0]$, $D = 1 - a - b - c$ — factors out the D magnification.

The magnification-factoring is a real, deliberate semantic change and is sound *for line-straightening*: POLY3’s $(1 - k_1)$ and PTLens’s D both scale the whole image so the corner stays put; dividing them out yields a pure $1 + \dots$ polynomial that straightens identically and differs only by an overall zoom a later crop absorbs. The doc comments state this. **Caveat:** because the magnification is dropped, the corrected image is at a *different* scale than `lensfun` would produce, so a side-by-side pixel comparison against `lensfun/darktable` will not match without re-zoom; and there is **no auto-scale-to-fill** (the stage keeps the source frame size — noted at `lib.rs:687`), so the residual zoom shows as black borders the user must crop. The `is_finite` guard against $k_1 \rightarrow 1 / D \rightarrow 0$ is appropriate.

VERDICT: Correct (model algebra and the superset polynomial). The coefficient form and the $\text{POLY3/POLY5/PTLENS} \rightarrow [d0..d3]$ mapping are right. Caveat: the magnification factoring intentionally differs from `lensfun`’s absolute scale; combined with the normalization-unit issue (§2.3) the absolute geometry will not match `lensfun`. **Citations:** `lensfun lfDistortionModel` (`docs/geometry-lensfun-lens-models.html`); PanoTools/Hugin lens correction model (`docs/geometry-panotools-lens-correction-model.html`); Brown, *Decentering Distortion of Lenses*, Photogrammetric Engineering 32(3), 1966.

5.2.2 2.2 Distortion direction — forward formula used as the backward (output→source) map

This is one of the two highest-risk questions, and the answer is **the code is correct given its coefficient mapping, by the same reasoning lensfun documents**.

lensfun’s own statement (“How the corrections work”, `docs/geometry-lensfun-corrections.html`): the model formulae “map the *undistorted* coordinate to the *distorted* coordinate ... This seems to be wrong at first because we want to undistort after all. But given how undistortion is actually done, it makes sense.” Undistortion is done by **inverse mapping**: for each *output (undistorted)* pixel you need the *source (distorted)* coordinate to sample — which is exactly the forward (undistorted→distorted) formula.

Crucial subtlety — lensfun does NOT apply the formula directly for poly3/poly5/ptlens distortion. In `mod-coord.cpp` the correction path (`Reverse/UnDist`) computes the *distorted* radius by **numerically inverting** the model with Newton’s method:

```
// ModifyCoord_UnDist_Poly3: Original function: Rd = k1_ * Ru^3 + Ru
```

```
// Target function: k1_ * Ru^3 + Ru -
Rd = 0 (solve for Ru by Newton)
```

(docs/geometry-lensfun-mod-coord.cpp:582-583, and similarly `_Poly5`, `_PTLens`.) lensfun treats the *output* pixel’s radius as R_d (the *measured/distorted* radius, because lensfun’s pipeline runs in the *captured* frame), solves for R_u , and the forward Dist functions (used for the opposite direction) apply $R_d = R_u(1 + \dots)$ directly.

latent instead treats the *output* pixel’s radius as r and **multiplies by $s(r)$ directly** to get the source radius (`Warp::map`). These are *opposite* algebraic operations on the *same* polynomial — but they correspond to *opposite definitions of which radius is the polynomial’s input*. latent’s choice is internally consistent and is the standard “PanoTools forward model used as the inverse-mapping lookup” (PanoTools’ $r_{\text{src}} = (\dots)r_{\text{dest}}$ is literally output→source). **The risk is not a sign error; it is that the polynomial input convention must match the coefficients’ convention.** PanoTools/Hugin coefficients are defined with r_{dest} (output) as the polynomial argument — so latent’s direct multiply is correct for PTLens-style coefficients. For lensfun POLY3/POLY5 coefficients, lensfun *defines* them with r_u (undistorted) as the argument and *inverts numerically*; applying them directly (no Newton inversion) is the **first-order-correct approximation** (it differs only at order k^2 and above), not the exact inverse. For the small k of real lenses this is a sub-pixel difference, but it is **not** bit-exact to lensfun.

VERDICT: Correct-with-caveats. Using the forward $s(r)$ as the OUTPUT→SOURCE map is the right *kind* of operation (it undistorts), and matches the PanoTools convention exactly. For lensfun POLY3/POLY5 it is a first-order approximation of lensfun’s Newton-inverted correction, not the exact same curve — acceptable for typical coefficients, worth a comment. There is **no direction (sign) bug**: a barrel lens ($k_1 < 0$ for POLY) pulls the corners in, straightening them, as the `lens_distortion_straightens_a_barrel_grid` test demonstrates. **Citations:** lensfun “How the corrections work” (docs/geometry-lensfun-corrections.html); `ModifyCoord_UnDist_{Poly3,Poly5,PTLens}` Newton inversion (docs/geometry-lensfun-mod-coord.cpp:560-741); PanoTools/Hugin model (docs/geometry-panotools-lens-correction-model.html).

5.2.3 2.3 Radius normalization unit — `lens_radial: inv_norm = 2/min(w,h)` vs lensfun’s half-diagonal

This is the single highest-risk silent-wrongness finding.

Code (latent-pipeline/src/lib.rs:672-677):

```
let inv_norm = 2.0 / w.min(h); // r = 1 at half the SHORTER side
```

and latent-edit/src/lib.rs:528-531 documents this as “the radius ... normalized by **half the shorter image side** (so $r = 1$ at the midpoint of the short edges, **matching lensfun**).”

lensfun’s actual normalization (libs/lensfun/modifier.cpp:79, downloaded docs/geometry-lensfun-modifier.cpp):


```
NormScale = hypot (36.0, 24.0) / Crop / hypot (Width + 1.0, Height + 1.0) /
↳ RealFocal;
```

This is **not** $2/\min(w, h)$. Reading the geometry from `GetAutoScale (mod-coord.cpp:492-495)`, the normalized distances of the eight test points are: - corners (4 of them): $\frac{1}{2}\sqrt{W^2 + H^2} \cdot \text{NormScale}$ - mid short-edge: $\frac{H}{2} \cdot \text{NormScale}$, mid long-edge: $\frac{W}{2} \cdot \text{NormScale}$.

Substituting `NormScale`, the **corner** normalized radius is

$$r_{\text{corner}} = \frac{1}{2}\sqrt{W^2 + H^2} \cdot \frac{\sqrt{36^2 + 24^2}}{\text{Crop} \cdot \sqrt{(W+1)^2 + (H+1)^2} \cdot \text{RealFocal}} \approx \frac{1}{2} \cdot \frac{d_{35}}{\text{Crop} \cdot \text{RealFocal}},$$

where $d_{35} = 43.27$ mm is the 35 mm diagonal. `lensfun`'s unit is therefore the **half-diagonal in focal-length-scaled 35 mm-equivalent units** — pixel-count-independent, anchored to the *diagonal* (corners), and *focal-length dependent*. In the canonical case it places $r \approx 1$ near the **image corner**, and the short-edge midpoint at $r = H/\sqrt{W^2 + H^2} < 1$.

`latent`'s $2/\min(w, h)$ instead places $r = 1$ at the **short-edge midpoint** and the corner at $r = \sqrt{W^2 + H^2}/\min(W, H)$ (e.g. 1.80 for 3:2, 1.60 for 4:3). So for the *same* physical point, `lensfun`'s r and `latent`'s r differ by a fixed ratio $k = \frac{\min(W, H)}{\text{diag-unit}}$. Because the distortion polynomial is in powers of r , every coefficient is silently rescaled by k^{-n} : d_1 (the r^2 term) is off by k^2 ($\approx 1.3\times$ for 3:2 even before the focal-length factor), d_3 by k^4 , etc. The result is **systematic over- or under-correction** that grows with radius and changes with aspect ratio — exactly the “silently rescales all coefficients” failure the doc comment claims to avoid.

Two compounding issues: - The doc comment's claim “matching `lensfun`” is **false**: `lensfun` uses the half-diagonal (focal-scaled), not half the shorter side. (Half-the-shorter-side is the **PanoTools/Hugin/PTGui** convention — “radius=1.0 is half the smaller side” — so the code matches *PanoTools*, not `lensfun`. If coefficients are sourced from Hugin's `abc` directly this is correct; if from `lensfun`'s database it is wrong.) - `lensfun`'s `NormScale` includes `/Crop/RealFocal`, i.e. a **focal-length scaling** the code has no analogue for. Even switching to half-diagonal would not reproduce `lensfun` without the `RealFocal/Crop` factor (which for rectilinear lenses where `RealFocal` $\approx$ `Crop` $\cdot$ `f_nominal` largely cancels against the sensor-vs-frame ratio, but not exactly, and notably not for fisheyes).

VERDICT: Questionable / Incorrect for `lensfun`-sourced coefficients. The normalization unit does not match `lensfun`; it matches *PanoTools*. Any coefficient pulled from the `lensfun` database (the stated runtime source) will be applied at the wrong radial scale, producing aspect-ratio-dependent over/under-correction. For coefficients authored against the *PanoTools* “half shorter side” convention it is correct. **Citations:** `lensfun NormScale` (`docs/geometry-lensfun-modifier.cpp:79`, `geometry-lensfun-mod-coord.cpp:492-495,521`); *PanoTools* normalization “radius 1.0 is half the smaller side” (`docs/geometry-panotools-lens-correction-model.html`).

Note on the optical center. `lensfun` scales the center offset by $\min(\text{Width}, \text{Height})/2 \cdot \text{NormScale}$ (`modifier.cpp:83-85`), i.e. `lensfun`'s `CenterX/Y` are a *shift from geometric center in the same normalized*

units. latent-lens maps center = [0.5 + CenterX, 0.5 + CenterY] (latent-lens/src/lib.rs:142) and lens_radial denormalizes to pixels via center[0]*(w-1). This treats CenterX/Y as a *fraction of the full width/height*, whereas lensfun's offset is a fraction of min(w,h)/2. For the overwhelmingly common centered case (CenterX=CenterY=0) this is exactly (0.5,0.5)→center and correct; for off-center calibrations it is mis-scaled by the same family of factors as §2.3. Low severity (rarely non-zero).

5.2.4 2.4 Lateral chromatic aberration (TCA) — per-channel radial scale, green fixed

Code (latent-pipeline/src/lib.rs:744, Warp::map_channel:272): channel_scale = [1 + ca[0], 1, 1 + ca[1]] (R and B scaled about the center, G = 1). ca_offsets (latent-lens/src/lib.rs:212) maps lensfun LINEAR/POLY3 TCA [k_R, k_B] → [k_R-1, k_B-1].

Authoritative (lensfun lfTCAModel, docs/geometry-lensfun-lens-models.html): LINEAR is $r_{d,R} = r_{u,R} k_R$, $r_{d,B} = r_{u,B} k_B$, with k_R, k_B defaulting to **1**. So the per-channel scale is k_R , and latent's "1 + ca[0]" with $ca[0] = k_R - 1$ reproduces k_R exactly. POLY3 TCA is $r_{d,R} = r_{u,R}(b_R r^2 + c_R r + v_R)$; ca_offsets keeps only the constant v_R (the leading interpolated term) and drops the radial dependence — an explicit, documented approximation.

- (a) **Lateral CA correction is a per-channel radial scaling** — confirmed; this is the textbook TCA model and exactly lensfun's LINEAR model. (b) **Green-as-reference is standard** — confirmed: green is the demosaic luminance reference and lensfun calibrates R and B *relative to* green (the TCA terms are k_R, k_B ; there is no k_G). Fixing $G = 1$ and scaling R/B is correct. The same normalization-unit issue (§2.3) applies to *where* the scaling happens radially, but since the LINEAR scale is *constant* in r , a normalization mismatch does **not** affect a pure-LINEAR TCA correction (a constant scale about the center is unit-independent). For POLY3 TCA the dropped radial terms *would* be unit-sensitive, but they are dropped anyway.

VERDICT: Correct (LINEAR TCA) / Correct-with-caveats (POLY3 reduced to its on-axis constant). Convention and green-reference match lensfun. The single-constant-scale-per-channel can leave residual CA on lenses whose CA varies strongly with radius (POLY3 calibrations), which the doc comment acknowledges. **Citations:** lensfun lfTCAModel LINEAR/POLY3 (docs/geometry-lensfun-lens-models.html).

5.2.5 2.5 Keystone / perspective homography — keystone_transform

Code (latent-pipeline/src/lib.rs:652-666): centered keystone K with bottom row $[a, b, 1]$, $a = \text{horizontal}/c_x$, $b = \text{vertical}/c_y$, sandwiched as $T(c) K T(-c)$ and pre-

baked into the explicit matrix. The output→source homography is

$$M = \begin{bmatrix} 1 + c_x a & c_x b & -c_x k \\ c_y a & 1 + c_y b & -c_y k \\ a & b & 1 - k \end{bmatrix}, \quad k = a c_x + b c_y.$$

Verification. This is a valid projective transform: bottom row $[a, b, 1 - k]$ is non-degenerate for the slider range, and the perspective divide $w = ax + by + (1 - k)$ varies the effective horizontal scale linearly with y (and vertical with x), which is precisely what converts a converging pencil of source lines into parallel output lines — the definition of keystone correction. The centering $T(c) K T(-c)$ keeps the frame center a fixed point (at (c_x, c_y) , $w = 1 - k + a c_x + b c_y = 1$, mapping to itself). The unit tests confirm: `keystone_straightens_converging_verticals` lifts two converging source points onto one output column; `keystone_zero_is_a_no_op` gives identity. The math is a standard 2-DOF perspective (one for each of vertical/horizontal convergence), equivalent to the “keystone” controls in Lightroom/Hugin.

The $w \leq 0$ **guard** (`map, lib.rs:137-142`; `Warp::map:252`) returns $(-1, -1)$ (outside the source → black). At extreme keystone a source plane point can pass through/behind the projection plane ($w \leq 0$), where $sx/w, sy/w$ would sign-flip or be $0/0$ (NaN). Returning a guaranteed-outside coordinate is the correct, standard handling (it is the projective “point at/behind infinity” case). `extreme_keystone_behind_the_plane_maps_outside` exercises it. **Minor caveat:** $(-1, -1)$ is only *just* outside; with bilinear it still reads pixel $(0, 0)$ at weight 0 via the `at()` bounds check → black, so it is safe in practice, but a value like `f32::NAN` or a sentinel far outside would be more obviously-outside. As written it is correct because the samplers bounds-check each tap.

VERDICT: Correct. Valid projective keystone, center-preserving, with correct behind-the-plane handling. **Citations:** Szeliski §2.1.2 (projective/homography transforms) and §3.6 (parametric warps), `docs/geometry-szeliski-03-image-processing.pdf`; Hardy/standard keystone derivation.

5.2.6 2.6 Rotation / straighten bounding-box expansion — `Transform::rotation`

Code (`latent-pipeline/src/lib.rs:110-128`): `new canvas nw = [w|cos| + h|sin|], nh = [w|sin| + h|cos|]`; the output→source matrix is the inverse rotation $\begin{bmatrix} \cos & \sin \\ -\sin & \cos \end{bmatrix}$ with a translation that maps the output center back to the source center.

Verification. The expanded-canvas formula $w|\cos \theta| + h|\sin \theta|$, $w|\sin \theta| + h|\cos \theta|$ is the **standard** axis-aligned bounding box of a rotated $w \times h$ rectangle (the extent of the rotated rectangle projected onto each axis) — correct, and it guarantees no content is clipped (the corner wedges fall outside the source and read black). The matrix is the *inverse* rotation (output→source for an inverse-mapped resample), with the bottom row $[0, 0, 1]$ (affine) — confirmed by `affine_constructors_have_a_unit_bottom_row`. The center-mapping offsets

m_{02}, m_{12} are built so the output center (dc_x, dc_y) maps to the source center (sc_x, sc_y); the “+0.5 / -0.5” terms correctly account for pixel-center sampling (integer index = pixel center). `straighten_expands_the_canvas_and_keeps_the_center` and `resample_rotation_expands_and_keeps_the_center` verify both the growth and that the center pixel is preserved to $< 10^{-4}$.

VERDICT: Correct. Standard rotated-bounding-box expansion and a correct center-preserving inverse-rotation map with proper pixel-center offsets. **Citations:** Szeliski §3.6.1 (rotation as a 2-D parametric transform; inverse warping), docs/geometry-szeliski-03-image-processing.pdf.

5.2.7 2.7 Resampling / interpolation — `sample_bilinear`, `warp`, `resample`

(a) Bilinear in LINEAR light — correct. Resampling is a reconstruction of the continuous signal; the signal whose samples are physically meaningful to average is **linear radiance**, not a gamma-encoded value. Averaging gamma-encoded values darkens edges and shifts hue. The whole pipeline is linear-light f32 and the `resample` inherits that. This is the correct choice and matches the consensus (Szeliski §3.6.1 treats resampling as filtering the image *signal*; gamma must be undone first — latent is already in the signal domain). **VERDICT: Correct.**

(b) Bilinear has no prefilter → aliasing on DOWNSCALE. `sample_bilinear` is a 2-tap interpolator: it reconstructs well for magnification and ~unit-scale maps but, on minification, it reads only the 4 nearest source pixels and **skips** the source pixels between output samples → undersampling/aliasing (moiré, jaggies). The code’s own comment (latent-cpu/src/lib.rs:269-272) acknowledges this and says downscaling is done separately by area-averaging (`ImageBuf::downscaled`). **The gap:** the geometry stage’s warps are *not* guaranteed unit-scale. A barrel-distortion correction *minifies* the corners (pulls them inward), a pincushion correction minifies the center, and a keystone correction strongly minifies the “far” (converging) edge — all done *through this same un-prefiltered bilinear*. There is no per-region MIP/area prefilter for the locally-minifying parts of a distortion/keystone map, so strongly-corrected regions will alias. Szeliski §3.5.2/§3.6 is explicit that **decimation must low-pass filter (prefilter) before subsampling**; a higher-order/anisotropic resampling filter (e.g. EWA, or at least a MIP-mapped trilinear) is the standard remedy for spatially-varying minification. **VERDICT: Correct-with-caveats (a real quality gap for strong distortion/keystone, not just whole-image downscale).**

(c) Single resample (compose all warps, interpolate once) — correct and best practice. Folding homography + radial + TCA into one `Warp::map` and sampling once is exactly Szeliski’s recommendation: chaining separate resampling passes compounds the interpolation blur, so production warpers compose the transform and resample once. **VERDICT: Correct.** This is the strongest part of the design.

(d) Border sampling reads black. `sample_bilinear`’s `at()` returns `[0,0,0]` for any out-of-bounds tap (latent-cpu/src/lib.rs:280), the WGSL fetch does the same, and the nearest-neighbor reference backends match. So sampling past the border fades to black — appropriate for geometry (rotated/distorted wedges and keystone

“sky” regions should be empty, to be cropped). One subtlety: a tap *one pixel outside* a valid edge blends edge-color toward black, producing a 1-px dark fringe at the image boundary. This is standard for “zero” border mode; “clamp/replicate” would avoid the fringe but is a defensible style choice (and the wedges are cropped anyway). **VERDICT: Correct (zero-border is a valid, intentional choice).** **Citations:** Szeliski, *Computer Vision: Algorithms and Applications*, §3.5.2 (decimation/prefiltering) and §3.6.1 (forward vs inverse warping, resampling, compositing transforms), docs/geometry-szeliski-03-image-processing.pdf; Heckbert, *Fundamentals of Texture Mapping and Image Warping* (MSc thesis, UC Berkeley 1989), §3–4 on resampling filters and prefiltering for minification.

5.2.8 2.8 Vignetting — correction (pre-resample reciprocal) and creative vignette (post-crop gain)

Code: correction at `apply_geometry` (lib.rs:697-708) builds `RadialGain { poly: l.vignetting, reciprocal: true }` and applies it **before** the resample. `RadialGain::at` (lib.rs:193) computes $p = 1 + g_0 r^2 + g_1 r^4 + g_2 r^6$ and returns $1/p$ when reciprocal. `vignetting_falloff` (latent-lens/src/lib.rs:225) passes `lensfun` PA terms straight through.

Authoritative (lensfun LF_VIGNETTING_MODEL_PA, docs/geometry-lensfun-lens-models.html): “Pablo D’Angelo vignetting model (a more general variant of the $\cos^4$ law): $C_d = C_s(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$ ” — where C_d is *corrected* and C_s is *source*. **Important nuance:** `lensfun`’s documented formula has the polynomial as a *multiplier on the source to get the corrected* value (the k are positive to brighten corners), i.e. `lensfun`’s “correction” multiplies by $(1 + k_1 r^2 + \dots)$. `latent` instead treats vignetting as the **measured falloff** (the captured brightness = ideal $\cdot (1 + \nu r^2 + \dots)$ with $\nu < 0$ for darker corners) and **divides** it out (reciprocal: true). These are reciprocal conventions: dividing by $(1 + \nu r^2)$ vs multiplying by $(1 + k r^2)$. To first order $1/(1 + \nu r^2) \approx 1 - \nu r^2$, so with $k = -\nu$ the two agree to first order but **not exactly** at higher order. The `latent-edit` doc comment (lib.rs:546-549) explicitly defines its convention as “captured brightness = ideal $\cdot (1 + \nu \dots)$, ν negative, correction divides it out” — which is self-consistent and matches the *sign* `lensfun`’s measured falloff has, but the *operation* (divide vs `lensfun-doc`’s multiply) differs at second order.

The model family ($\cos^4$ /polynomial in r^2) is correct: the PA model is the standard generalization of $\cos^4$ falloff and uses exactly $1 + k_1 r^2 + k_2 r^4 + k_3 r^6$ — matching the code’s poly in r^2 .

Pre-resample correction is correct. Vignetting is a property of the *captured* (pre-geometry) frame; flat-fielding must happen in source coordinates before any geometric resampling moves pixels — which is what `lensfun` does (its pipeline order is colour/vignetting then geometry) and what `apply_geometry` does (vignetting multiply, then warp). Doing it as a *multiply* (not an interpolation) is also correct — it adds no blur. **VERDICT: Correct (model family, sign, pre-resample placement); Correct-with-caveats on the divide-vs-multiply convention** — verify whether `lensfun`’s interpolated PA terms are meant to be *divided out* (as `latent` does) or

multiplied in. If the Terms are the falloff, divide is right; if they are already the correction gain, latent would be inverting twice. The unit-normalization issue (§2.3) also rescales r here.

Creative vignette (lib.rs:756-767): a RadialGain about the post-crop center with $\text{inv_norm} = 2/\sqrt{w^2+h^2}$ (so $r = 1$ at the **corners** — the diagonal), $\text{poly} = [\text{amount}, 0, 0]$, $\text{reciprocal} = \text{false}$ (a direct multiply). This is geometrically right for a creative vignette (corners reach full effect, $\cos^4$ -like r^2 falloff), and `creative_vignette_darkens_corners_and_keeps_center` confirms center untouched / corners darkened. Worth noting the **internal inconsistency**: the *creative* vignette normalizes to the **half-diagonal** (corners at $r = 1$) — which is the *correct lensfun-style* normalization — while the *lens distortion/vignetting correction* normalizes to the half-shorter-side (§2.3). The creative one is self-contained so this is fine, but it shows the half-diagonal unit was available and is arguably what §2.3 should also use.

VERDICT: Correct (creative vignette; correction model family & placement) / Correct-with-caveats (divide-vs-multiply convention for the PA correction). Citations: `lensfun lfVignettingModel PA` (`docs/geometry-lensfun-lens-models.html`); D’Angelo, *Radiometric alignment and vignetting calibration*, ICVS 2007.

5.2.9 2.9 GPU vs CPU resample equivalence — `resample.wgsl`

Code: the WGSL compute shader (`latent-gpu/src/resample.wgsl`) applies the same row-major homography with the perspective divide ($w = m6 \cdot ox + m7 \cdot oy + m8$; $sx = (\dots)/w$), the same floor/frac bilinear with mix, and the same out-of-bounds→black fetch.

Verification. Same inverse map, same bilinear weights, same zero-border. **Two discrepancies vs the CPU/Warp path:** 1. **No $w \leq 0$ guard.** `Transform::map` and `Warp::map` return $(-1, -1)$ when $w \leq 0$ (`lib.rs:137, :252`); the shader divides unconditionally ($sx = (\dots)/w$). For $w < 0$ the GPU gets a sign-flipped (possibly in-bounds!) coordinate and may sample *real* source pixels instead of black; for $w = 0$ it produces $\pm\infty/\text{NaN}$, and `i32(floor(inf))` is UB-ish → likely reads black, but not guaranteed to match the CPU’s deterministic $(-1, -1)$. On extreme keystone the GPU and CPU outputs can **differ** at the behind-the-plane corners. 2. **The WGSL is resample only — it has no warp (radial distortion / TCA) analogue.** So whenever a lens distortion or CA term is present, the GPU path either falls back to CPU or is simply not covered here; the shader handles only the pure-homography case. (Confirm the dispatch falls back to CPU warp for lens terms — if it silently runs the homography-only shader, lens distortion/CA would be dropped on GPU.)

Otherwise the homography resample matches the CPU bit-for-bit modulo float-order (same operations).

VERDICT: Correct-with-caveats. The homography bilinear resample matches CPU. Missing: the $w \leq 0$ guard (extreme-keystone divergence) and any radial/TCA warp shader (the Warp path is CPU-only here). The header comment claims parity with the

CPU resample; that parity holds only for $w > 0$ homographies. **Citations:** comparison of latent-gpu/src/resample.wgsl:46-48 vs latent-pipeline/src/lib.rs:137-142,251-255.

5.3 3. Findings by Severity

5.3.1 CRITICAL

C1 — Distortion/vignetting/TCA radius normalization unit does not match lensfun (silent coefficient rescale). latent-pipeline/src/lib.rs:675 ($\text{inv_norm} = 2.0 / w.\text{min}(h)$) and the convention asserted at latent-edit/src/lib.rs:528-531 (“half the shorter image side ... matching lensfun”). lensfun normalizes by the **half-diagonal** in focal-scaled 35 mm units ($\text{NormScale} = \text{hypot}(36,24)/\text{Crop}/\text{hypot}(W+1,H+1)/\text{RealFocal}$ docs/geometry-lensfun-modifier.cpp:79; corner at $r \approx 1$ per mod-coord.cpp:492-495), **not** half the shorter side. Half-the-shorter-side is the **PanoTools/Hugin** convention. Result: every lensfun-sourced distortion coefficient is applied at the wrong radial scale (off by k^n where $k = \min(w,h)/\text{diag-unit}$, $\approx 1.3\text{--}1.8\times$ per even order for common aspect ratios), giving aspect-ratio-dependent over/under-correction that worsens toward the corners. The vignetting correction (§2.8) and POLY3-TCA radial terms inherit the same rescale. **Reference:** docs/geometry-lensfun-modifier.cpp:79, geometry-lensfun-mod-coord.cpp:492-495,521; PanoTools “radius 1.0 is half the smaller side” (docs/geometry-panotools-lens-correction-model.html). **Recommendation:** Decide the source-of-truth convention and make the doc comment honest. If coefficients come from **lensfun**: normalize by the half-diagonal and incorporate the RealFocal/Crop scaling (set $\text{inv_norm} = 2.0/\text{hypot}(w,h)$ as the geometric baseline, and ideally fold the focal-length factor so it matches NormScale). If coefficients come from **PanoTools/Hugin abc** directly, the current $2/\min(w,h)$ is correct and only the “matching lensfun” comment is wrong. Add a round-trip test that straightens a known-distorted grid using a real lensfun <distortion> entry and checks residual line-straightness vs lensfun’s own output.

5.3.2 HIGH

H1 — GPU resample.wgsl lacks the $w \leq 0$ guard and has no radial/TCA warp shader. latent-gpu/src/resample.wgsl:46-48 divides by w unconditionally (CPU returns $(-1,-1)$ for $w \leq 0$, lib.rs:137), so extreme-keystone corners can sample real pixels or NaN on GPU and diverge from CPU; and the shader implements only the homography resample, not the Warp (distortion + CA) path. If the dispatcher does not fall back to CPU warp when a lens term is present, lens distortion/CA would be silently dropped on GPU. **Reference:** §2.9; resample.wgsl vs latent-pipeline/src/lib.rs:137-142,251-267,272-282. **Recommendation:** Add `if (w <= 0.0) { out = black; return; }` to the shader; add a warp.wgsl mirroring `Warp::map/map_channel` (radial Horner + per-channel scale), or assert a CPU fallback for any non-trivial Warp. Add a CPU/GPU equivalence test over a perspective + distortion case.

5.3.3 MEDIUM

M1 — Forward distortion formula applied directly is only first-order-equivalent to lensfun’s Newton-inverted correction (for POLY3/POLY5). latent-pipeline/src/lib.rs:264-267 multiplies the output radius by $s(r)$; lensfun’s `UnDist_{Poly3,Poly5}` solve $R_d = R_u(1 + \dots)$ for R_u by Newton (docs/geometry-lensfun-mod-coord.cpp:582-583). For PanoTools/PTLens coefficients the direct multiply is the *defined* operation (output→source), so this is exact; for lensfun POLY coefficients it differs at $O(k^2)$. Sub-pixel for real lenses but not bit-exact. **Reference:** §2.2. **Recommendation:** Note this in the doc comment; if exactness vs lensfun is wanted for POLY models, Newton-invert (2-3 iterations) instead of the direct multiply.

M2 — Vignetting correction divides by the falloff polynomial where lensfun’s PA doc multiplies (reciprocal-convention ambiguity). latent-pipeline/src/lib.rs:705 (`reciprocal: true`) divides by $(1 + vr^2 + \dots)$; lensfun’s documented PA correction is $C_d = C_s(1 + k_1r^2 + \dots)$ (a multiply). The two agree only to first order. `vignetting_falloff` passes lensfun’s interpolated Terms straight through (latent-lens/src/lib.rs:225) — so the question is whether those Terms are the *falloff* (divide is right) or the *correction gain* (then latent double-inverts). **Reference:** §2.8; docs/geometry-lensfun-lens-models.html (PA model). **Recommendation:** Confirm against lensfun’s `ModifyColor/vignetting` apply code whether the stored PA k are falloff or gain, and align the divide/multiply (and sign) accordingly. Add a flat-field round-trip test against a known lensfun vignetting entry.

M3 — Un-prefiltered bilinear aliases on the locally-minifying parts of distortion/keystone maps. latent-cpu/src/lib.rs:273 (`sample_bilinear`). The comment scopes bilinear to $\sim$ unit-scale and defers downscale to area-averaging, but distortion/keystone warps minify locally (corners under barrel correction, the converging edge under keystone) through this same 2-tap path → aliasing with no prefilter. **Reference:** §2.7(b); Szeliski §3.5.2/§3.6 (prefilter before subsampling). **Recommendation:** For strong corrections, use a MIP-mapped/trilinear or EWA sampler in the warp, or at minimum a small adaptive box prefilter sized by the local Jacobian of the map. Low urgency if corrections are mild, but document the limitation in warp.

5.3.4 LOW

L1 — Optical-center offset scaling differs from lensfun for off-center calibrations. latent-lens/src/lib.rs:142 maps `center = 0.5 + CenterX` (fraction of full dimension); lensfun’s `CenterX/Y` are offsets in $\min(w,h)/2 \cdot \text{NormScale}$ units (`modifier.cpp`:83-85). Exact for the common centered case (`CenterX=0`), mis-scaled otherwise. **Recommendation:** Scale the center offset by $\min(w,h)/2$ to match lensfun if off-center profiles are ever used.

L2 — $w \leq 0$ sentinel (-1, -1) is only marginally outside. latent-pipeline/src/lib.rs:141,254. Safe because every sampler bounds-checks each tap, but a more obviously-outside sentinel (e.g. a large negative) would be more robust to future sampler changes. **Recommendation:** Optional; keep the bounds-checks as the real guarantee.

5.3.5 NOTE

N1 — No auto-scale-to-fill. `apply_geometry` keeps the source frame size (`lib.rs:687`), so distortion/keystone/straighten leave black wedges to be cropped. Intentional and documented; `lensfun` offers `GetAutoScale` (`mod-coord.cpp:469`) for this — a future addition.

N2 — Magnification factored out of POLY3/PTLens. `latent-lens/src/lib.rs:178-182` deliberately drops POLY3's $(1 - k_1)$ and PTLens's D overall scale. Correct for line-straightening; means the absolute output scale differs from `lensfun` (a crop/zoom absorbs it). Documented.

N3 — Single-resample design is exemplary. Folding homography $\circ$ radial $\circ$ per-channel-CA into one Warp and interpolating once (`lib.rs:726-746`, `latent-cpu/src/lib.rs:118-143`) is the correct best practice (Szeliski §3.6.1) and the strongest part of the geometry stage.

5.4 4. References

Primary — lensfun (downloaded under docs/): - `lensfun` API, *Structures and functions for lenses* — `lfDistortionModel` (POLY3/POLY5/PTLENS), `lfTCAModel` (LINEAR/POLY3), `lfVignettingModel` (PA). `docs/geometry-lensfun-lens-models.html` — https://lensfun.github.io/manual/latest/group__Lens.html - `lensfun`, *How the corrections work* (forward-formula / backward-mapping direction). `docs/geometry-lensfun-corrections.html` — <https://lensfun.github.io/manual/latest/corrections.html> - `lensfun`, *Lens calibration data format*. `docs/geometry-lensfun-calibration-format.html` — https://lensfun.github.io/manual/latest/element_calibration.html - `lensfun` source, `libs/lensfun/modifier.cpp` — `NormScale = hypot(36,24)/Crop/hypot(W+1,H+1)/RealFocal` and center scaling. `docs/geometry-lensfun-modifier.cpp` — <https://github.com/lensfun/lensfun/blob/master/libs/lensfun/modifier.cpp> - `lensfun` source, `libs/lensfun/mod-coord.cpp` — `ModifyCoord_UnDist_{Poly3,Poly5,PTLens}` (Newton inversion), `GetAutoScale`, `ApplyGeometryDistortion`. `docs/geometry-lensfun-mod-coord.cpp` — <https://github.com/lensfun/lensfun/blob/master/libs/lensfun/mod-coord.cpp>

Primary — PanoTools / resampling: - `PanoTools` / Hugin, *Lens correction model* (the abc model, $d = 1 - (a + b + c)$, “radius 1.0 is half the smaller side”). `docs/geometry-panotools-lens-correction-model.html` — https://hugin.sourceforge.io/docs/manual/Lens_correction_model.html - R. Szeliski, *Computer Vision: Algorithms and Applications*, §3.5.2 (decimation/prefiltering), §3.6.1 (forward vs inverse warping, resampling, compositing transforms). `docs/geometry-szeliski-03-image-processing.pdf` — <http://mesh.brown.edu/engn1610/szeliski/ImageProcessing.pdf> (2nd ed. full text: <https://szeliski.org/Book/>) - P. Heckbert, *Fundamentals of Texture Mapping and Image Warping*, MSc thesis, UC Berkeley, 1989 (resampling filters; prefiltering for minification; EWA) — <https://www.cs.cmu.edu/~ph/texfund/texfund.html>

Background: - D. C. Brown, *Decentering Distortion of Lenses*, *Photogrammetric Engineering* 32(3):444-462, 1966 (even-powered radial model). - P. D’Angelo, *Radiometric alignment and vignetting calibration*, ICVS 2007 (the PA vignetting model).